

Course Project
CCSW 315 Software Process Models
Academic Year: Fall 2025
Final report



Atelier

Group 3		
Student Name	Section	Id
Shumokh Alsharif	SE3	2311165
Reman Alamoudi	SE3	2312582
Ghaida Almehmadi	SE1	2317174
Remas Alsulami	SE3	2310961
Marwa Hadaddi	SE	2310271

Important links:

[Atelier Website](#)

[GitHub Repository](#)

[Design \(Figma\)](#)

[Agile planning \(Trello\)](#)

Table of Contents

Sprint	06
Project Description Backlog	06
Problem definition	6
Proposed Solution	6
Scope	6
Users	6
Product Backlog	7
Functional Requirements	7
Non-functional Requirements	8
Use Case Diagram	9
Class Diagram	10
Tools To be Used	11
 Sprint 1	 12
Backlog	12
Components Name	12
Functional Requirements	12
Non-Functional Requirements	12
Diagrams	13
Use case	13
Sequence	13
Stories	14
Browse main feed	14
Browse collection	14
View product details	14
Meetings	15
Implementation in Web browser (UI):	21
Splash screen	21
Home page (collections)	21
Home page (Suggestions)	22
Collections filter	22
Collection page	24
product details page	25

Loading skeleton	25
Implementation in mobile browser (UI):.....	26
Splash screen.....	26
Home page (collections).....	26
Home page (Suggestions).....	26
Collections filter.....	26
Collection page.....	27
product details page.....	28
Loading skeleton	28
Test Cases	29
Browse main feed.....	29
Browse collection	29
View product details	30
 Sprint 2.....	 31
Backlog	31
Components Name	31
Functional Requirements.....	31
Non-Functional Requirements.....	31
Diagrams	32
Use case.....	32
Sequence.....	33
Stories	38
Meetings	40
Implementation in Web browser (UI):	43
Welcome page	43
Sign up page	44
Log in page	44
Profile page	45
Wishlist pages	46
Commission pages	47
Implementation in mobile browser (UI):.....	49
Welcome page	49
Sign up page	49
Log in page	49

Profile page	49
Wishlist pages	50
Commission pages	50
Test Cases	52
Log In.....	52
Register.....	53
Wishlist page	54
Commission.....	55
Artist profile	56
 Sprint 3.....	 58
Backlog	58
Components Name	58
Functional Requirements.....	58
Non-Functional Requirements.....	58
Diagrams	59
Use case.....	59
Sequence.....	60
Stories	62
Meetings	63
Implementation in Web browser (UD):	65
AR button positioned above the product image	65
AR Feature	66
Add to cart button on product page	66
Delete from the cart button on product page	67
Cart page	67
Checkout page.....	68
Implementation in mobile browser (UD):.....	69
AR button positioned above the product image	69
AR Feature	70
Add to cart button on product page	71
Delete from the cart button on product page	71
Cart page	72
Checkout page.....	73
Test Cases	74

Cart.....	74
Checkout.....	75
Web AR.....	75
Code.....	76
Backend	76
Server.....	76
AR	90
Frontend.....	91

Sprint 0

Project Description

This project is about building a web platform that brings together artists, galleries, collectors, and people who love art. The platform will make it easy for artists to show their work, and for buyers to find, buy, or even request custom artworks. It is made to improve access and communication in the art community.

Problem definition

Today, many artists use social media or small local shows to share their work. This limits how many people see their art and lowers their chances of selling. Customers also find it hard to compare artworks or ask for special orders in a clear way. Because there is no one platform for both sides, artists and buyers often miss chances to connect.

Proposed Solution

The solution is to make a unified art platform that is simple and helpful for both artists and buyers. It will show products in a clear way, support safe online payments, and let customers save or request items. Artists will also have profile pages with information about them and their work. This will make it easier for buyers to trust and connect with them. Overall, the platform will help both sides by opening new chances for buying, selling, and working together in art.

Scope

A web platform for art enthusiasts, collectors, and galleries to browse artisan products, place orders, pre-orders, or commissions, and explore artisan profiles. The system includes a shopping cart, multiple payment options, and works on different devices and browsers.

Users

- Art Enthusiasts
- Art Collectors
- Galleries / Organizations
- Artisans

Product Backlog

Functional Requirements:

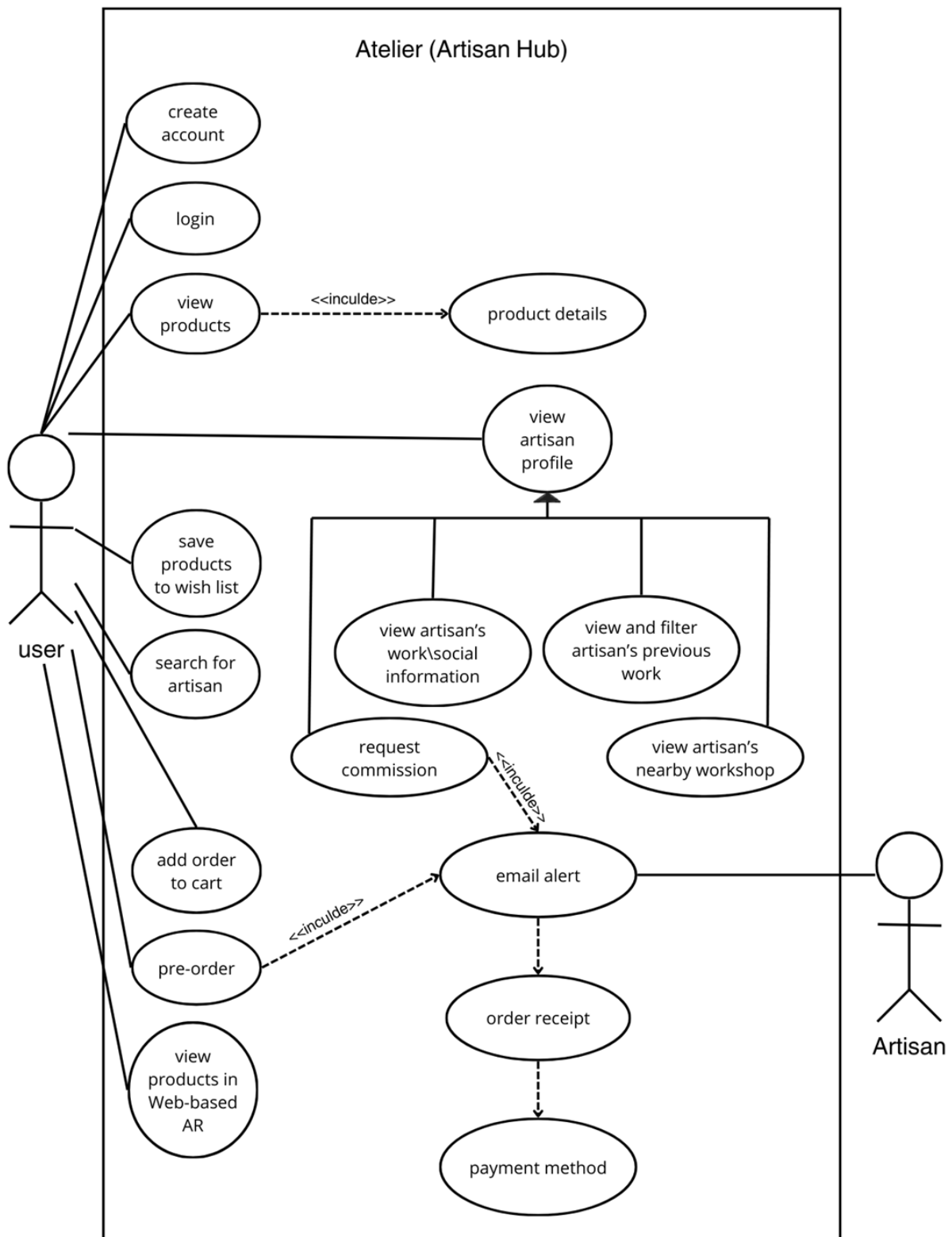
Req-ID	Description	Priority	Comment
FR-1	The system shall allow users to register and log in to their accounts	Low	
FR-2	The system shall display products in a card format within the main feed, with each card containing: - product photos. - The product price. - A description preview. - An "Add to Cart" button. - The artisan's profile picture and username	High	
FR-3	The system shall allow users to filter the main feed by category	High	
FR-4	The system shall provide a detailed product page which shall display: - All elements in FR-1 - A full product description - Detailed product dimensions. - Care instructions and cautions.	High	
FR-5	The system shall allow users to add products to a shopping cart.	High	
FR-6	The system shall group products by their respective shop (artisan) in the cart	Medium	
FR-7	The system shall calculate and display a subtotal for the items within each shop's grouping separately.	Low	A general total might be easier for users
FR-8	The system shall calculate the total of the cart.	High	
FR-9	The system shall provide a checkout process with the following payment methods: Apple Pay, debit card, credit card.	High	
FR-10	The system shall allow users to save products to a personal wishlist.	Low	
FR-11	The system shall allow users to navigate to an artisan's full profile by selecting the artisan's username	High	
FR-12	The system shall display an artisan's profile page containing: - The artisan's profile picture, and name. - A bio "About the Artist" - Links to the artisan's social media - A "Request Commission" button	High	

	- A gallery of the artisan's previous works.		
FR-13	The system shall provide filters for viewing artisan's work gallery "All Works" and "Available Works."	Low	
FR-14	The system shall provide a form for users to submit custom commission requests to an artisan, with the following fields: product type, dimensions, and a detailed description and attached media.	High	
FR-15	The system shall automatically send an email alert to the artisan upon receiving a commission request	Medium	
FR-16	The system shall allow users to place orders for products within a pre-order collection	Low	
FR-17	The system shall implement a Web-based Augmented Reality (Web AR) feature to allow users to visualize products in their environment.	Medium	Compatibility challenges

Non-functional Requirements:

Req-ID	Description	Priority	Comment
NFR-1	The main feed should load and be usable within 5 seconds	High	
NFR-2	The website should be useable on both mobile phones and desktop computers.	High	
NFR-3	User data, product information, and orders must be saved correctly and not be lost after a page refreshing	High	
NFR-4	The system shall maintain full functionality on Chrome, Firefox, Safari, and Edge.	Medim	
NFR-5	The system should support a minimum of 500 concurrent users performing typical activities	High	
NFR-6	The code should be consistently formatted and well documented	Medium	
NFR-7	The system data must be stored securely in the database	High	
NFR-8	Product images shall be optimized to minimize load time of 5 seconds	Low	
NFR-9	The system shall display images with minimum of 200PPI	High	

Use Case Diagram:



Tools To be Used:

Tool / Technology Purpose	
Visual Studio Code	IDE for writing frontend (HTML, CSS, JS) and backend code.
Git + GitHub	Version control for collaboration, commits, branches, and tracking changes.
Trello	Agile planning tool to manage sprint backlog, tasks, and progress.
Figma	Modeling/design tool for UI/UX mockups, wireframes, and style guide.
WhatsApp	Messaging & team coordination during development.
mongoDB	Database to store products, artisan profiles, orders; also provides authentication.
HTML5	Structure of web pages (frontend).
CSS3	Styling of web pages (colors, layout, responsiveness).
JavaScript (Vanilla)	Frontend interactivity, form validation.
Node.js	For backend server
Microsoft word	For documentation
Google meet	For virtual meetings

Sprint 1

1 Backlog (Name of functional requirement)

1.1 Components Name:

Browse main feed, browse collection, view product details

1.2 Functional Requirements:

FR-2: The system shall display products in a card format within the main feed

FR-3: The system shall allow users to filter the main feed by category

FR-4: The system shall provide a detailed product page which shall display

- Product photo
- The product price.
- An "Add to Cart" button.
- A full product description
- Detailed product dimensions.
- Care instructions and cautions.
- The artisan's profile picture and username

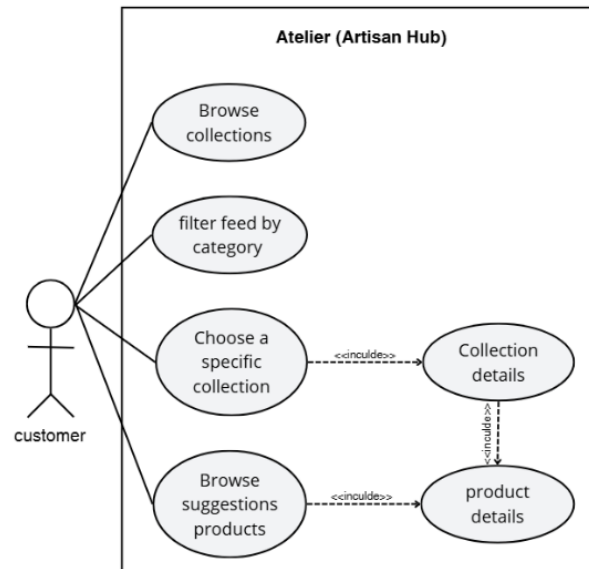
1.3 Non-Functional Requirements:

NFR-2: The website should be useable on both mobile phones and desktop computers

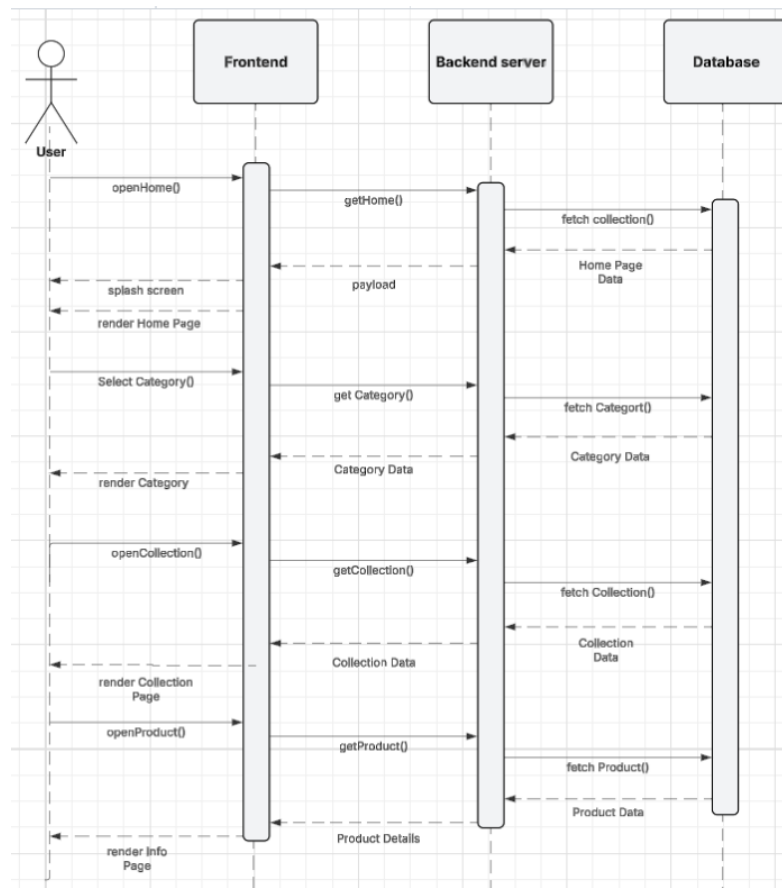
NFR-4: The system shall maintain full functionality on Chrome, Firefox, Safari, and Edge.

2 Diagrams

2.1 Use case:



2.2 Sequence Diagram:



3 Stories:

3.1 Browse main feed

Component Name: Browse main feed

Story Name: Browse main feed

Story Sequence No: 01

Story Short Description: Users can browse the available collections and products.

Story Long Description: Users shall be able to browse products and featured collections which can be filtered according to the user's desired category. The data shall be sent from the database and displayed randomly to maximize product discovery for the user.

3.2 Browse collection

Component Name: Browse collection

Story Name: Browse collection

Story Sequence No: 02

Story Short Description: The system will display collection information.

Story Long Description: The user will navigate to the collection page when he clicks on a specific collection. The page will contain:

- collection name
- artist information (name, username and profile picture)
- all the products within the collection.

3.3 View product details

Component Name: View product details

Story Name: View product details

Story Sequence No: 02 or 03

Story Short Description: The system shall display product details.

Story Long Description: The user will navigate to the product page when he clicks on a specific product from the home page or the collection page. The page will display:

- the artist information (name, username, profile picture)
- product name
- Price
- dimensions
- category
- description

4 Meetings:

Sprint Meeting(s)

Project Name: Atelier

Project Members:

Remas Alsulami - Marwa Hadaddi - Reman Alamoudi - Ghaida Almehmadi - Shumokh Alsharif

Sprint #1 *Stand up Meeting* [21 Sep 2025]

Sprint Duration: 3 weeks

Scrum Master: Shumokh Alsharif

Client: Malak Alharbi

Pair Programmers:

Remas Alsulami - Marwa Hadaddi - Reman Alamoudi - Ghaida Almehmadi - Shumokh Alsharif

Stories:

Components Name:

- Browse main feed
- Browse collection
- View product details

Follow-up meeting questions:

1. What has been completed since the last meeting?
2. What are you going to be working on next?
3. Do you have any issues/impediments?

Scrum's Master comments based on the above questions:

- 1- The high-fidelity design is completed and ready to hand off to developers
- 2- Next is:
 - Build the back-and server structure
 - Initiate the database with test data
 - Start with the front-end skeleton
- 3- So far none

Sprint Meeting(s)

Project Name: Atelier

Project Members:

Remas Alsulami - Marwa Hadaddi - Reman Alamoudi - Ghaida Almehmadi - Shumokh Alsharif

Sprint #1 *progress check in* **[25 Sep 2025]**

Sprint Duration: 3 weeks

Scrum Master: Shumokh Alsharif

Client: Malak Alharbi

Pair Programmers:

Remas Alsulami - Marwa Hadaddi - Reman Alamoudi - Ghaida Almehmadi - Shumokh Alsharif

Stories:

Components Name:

- Browse main feed
- Browse collection
- View product details

Follow-up meeting questions:

1. What has been completed since the last meeting?
2. What are you going to be working on next?
3. Do you have any issues/impediments?

Scrum's Master comments based on the above questions:

- 1- Initial pages are done, server and database successfully connected
- 2- Next is:
 - Create the actual endpoint
 - Add responsive grid for home and collection pages
 - Switch front-end from hard coded data to fetching a Json file
 - Test server with postman
- 3- Coordinating between each commit, push, pull to the repo

Sprint Meeting(s)

Project Name: Atelier

Project Members:

Remas Alsulami - Marwa Hadaddi - Reman Alamoudi - Ghaida Almehmadi - Shumokh Alsharif

Sprint #1
progress check in
[29 Sep 2025]

Sprint Duration: 3 weeks

Scrum Master: Shumokh Alsharif

Client: Malak Alharbi

Pair Programmers:

Remas Alsulami - Marwa Hadaddi - Reman Alamoudi - Ghaida Almehmadi - Shumokh Alsharif

Stories:

Components Name:

- Browse main feed
- Browse collection
- View product details

Follow-up meeting questions:

1. What has been completed since the last meeting?
2. What are you going to be working on next?
3. Do you have any issues/impediments?

Scrum's Master comments based on the above questions:

- 1- **Front-end pages have all required elements, all back-end server endpoints work and respond correctly**
- 2- **Next is:**
 - Deploy the server
 - Connect the front-end with the back-end
 - Populating the database with real data
- 3- **The server denied front-end access so we couldn't load data from the database**

Sprint Meeting(s)

Project Name: Atelier

Project Members:

Remas Alsulami - Marwa Hadaddi - Reman Alamoudi - Ghaida Almeahmadi - Shumokh Alsharif

Sprint #1
progress check in
[2 Oct 2025]

Sprint Duration: 3 weeks

Scrum Master: Shumokh Alsharif

Client: Malak Alharbi

Pair Programmers:

Remas Alsulami - Marwa Hadaddi - Reman Alamoudi - Ghaida Almeahmadi - Shumokh Alsharif

Stories:

Components Name:

- Browse main feed
- Browse collection
- View product details

Follow-up meeting questions:

1. What has been completed since the last meeting?
2. What are you going to be working on next?
3. Do you have any issues/impediments?

Scrum's Master comments based on the above questions:

- 1- **Front-end is successfully connected to back-end, server is live**
- 2- **Next is:**
 - Add navigation between pages
 - Fix the fetching issues
 - Add an endpoint that lists all collections names
 - Adding a splash screen
- 3- **The collections endpoint needs improvement (provide the names listed & be filtered), artist data are not being fetched**

Sprint Meeting(s)

Project Name: Atelier

Project Members:

Remas Alsulami - Marwa Hadaddi - Reman Alamoudi - Ghaida Almeahmadi - Shumokh Alsharif

Sprint #1
progress check in
[5 Oct 2025]

Sprint Duration: 3 weeks

Scrum Master: Shumokh Alsharif

Client: Malak Alharbi

Pair Programmers:

Remas Alsulami - Marwa Hadaddi - Reman Alamoudi - Ghaida Almeahmadi - Shumokh Alsharif

Stories:

Components Name:

- Browse main feed
- Browse collection
- View product details

Follow-up meeting questions:

1. What has been completed since the last meeting?
2. What are you going to be working on next?
3. Do you have any issues/impediments?

Scrum's Master comments based on the above questions:

- 1- **Fetching artist data now works, the new endpoint is completed, navigation between pages is done**
- 2- **Next is:**
 - Complete the database
 - Merge the splash screen branch
 - Ensure visual consistency across all pages
 - Fix the painting and pottery category problem
- 3- **The intro keeps repeating every time the home page is loaded, not all categories are displayed**

Sprint Meeting(s)

Project Name: Atelier

Project Members:

Remas Alsulami - Marwa Hadaddi - Reman Alamoudi - Ghaida Almeahmadi - Shumokh Alsharif

Sprint #1
progress check in
[9 Oct 2025]

Sprint Duration: 3 weeks

Scrum Master: Shumokh Alsharif

Client: Malak Alharbi

Pair Programmers:

Remas Alsulami - Marwa Hadaddi - Reman Alamoudi - Ghaida Almeahmadi - Shumokh Alsharif

Stories:

Components Name:

- Browse main feed
- Browse collection
- View product details

Follow-up meeting questions:

1. What has been completed since the last meeting?
2. What are you going to be working on next?
3. Do you have any issues/impediments?

Scrum's Master comments based on the above questions:

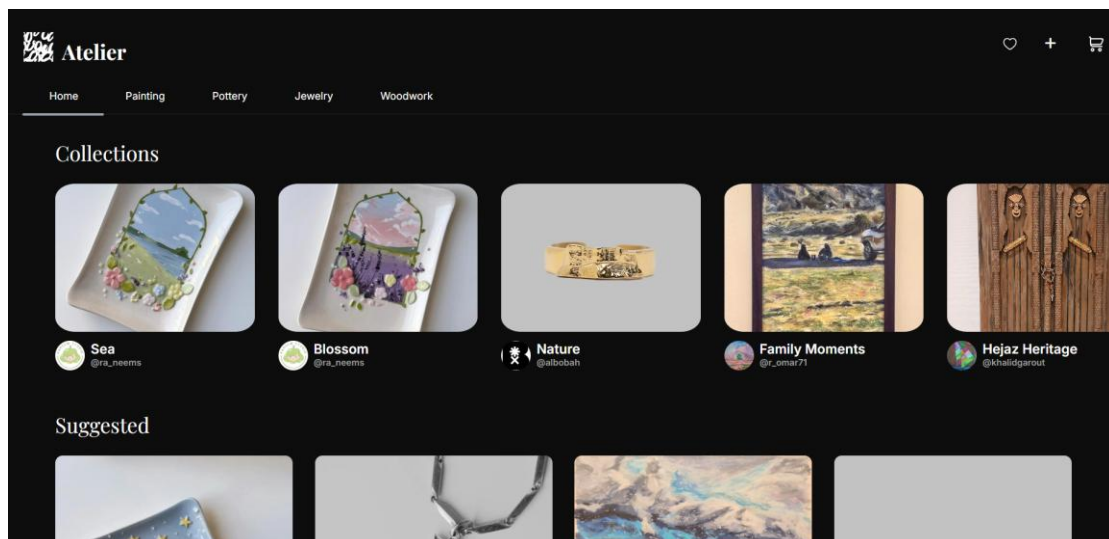
- 1- All coding tasks are completed and the website fully functioning
- 2- Next is:
 - Deploy the website
 - Test on different devices
 - Complete sprint documentation
- 3- So far none

5 Implementation in Web browser (UI):

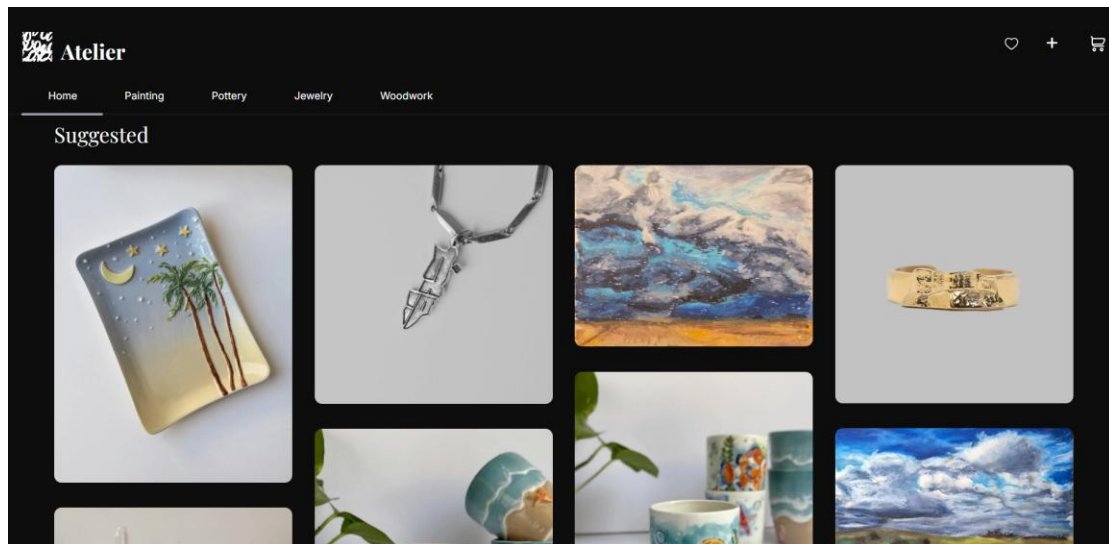
5.1 Splash screen



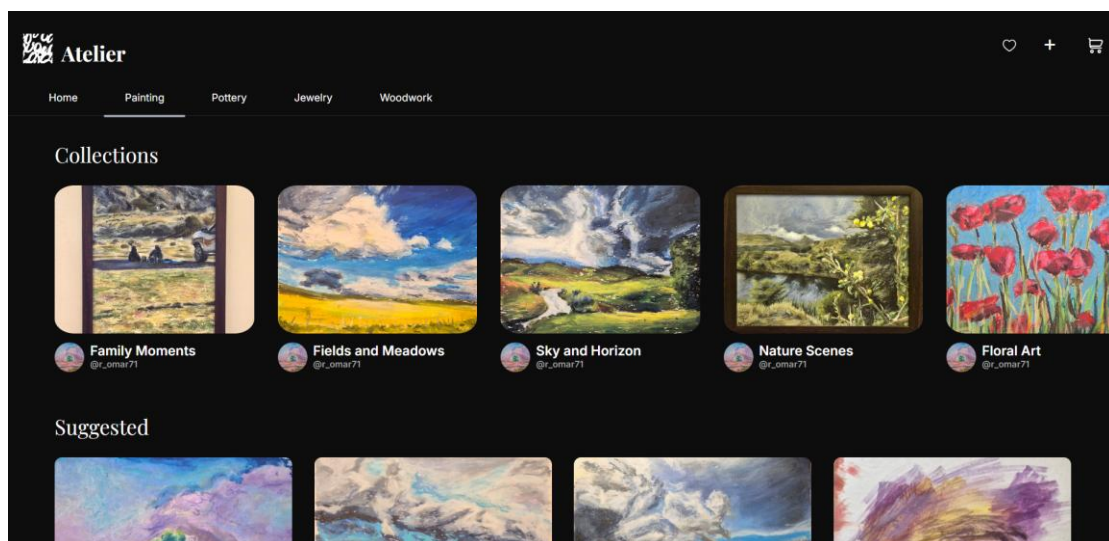
5.2 Home Page (collections)



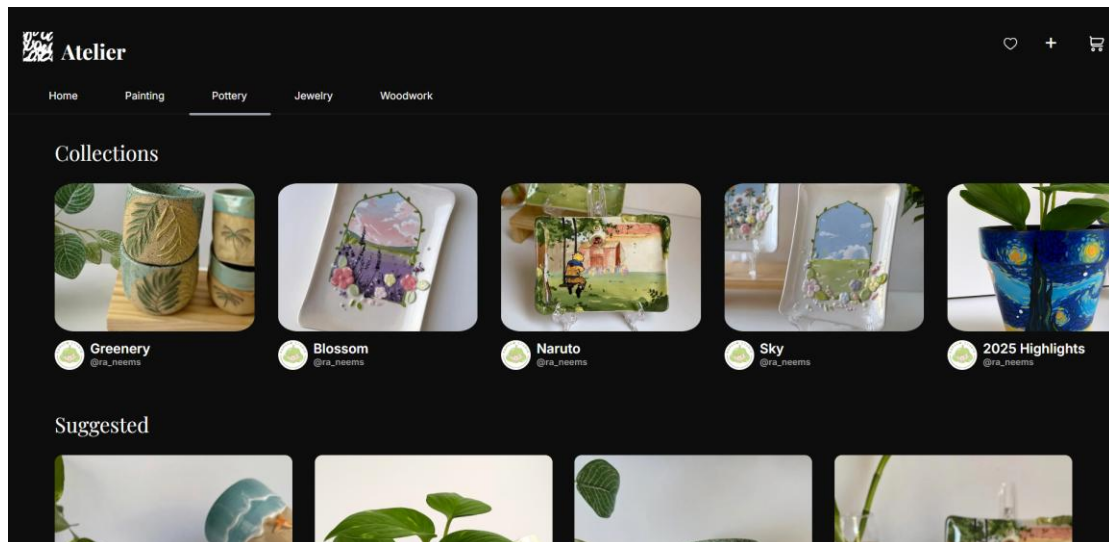
5.2 Home Page(suggestions)



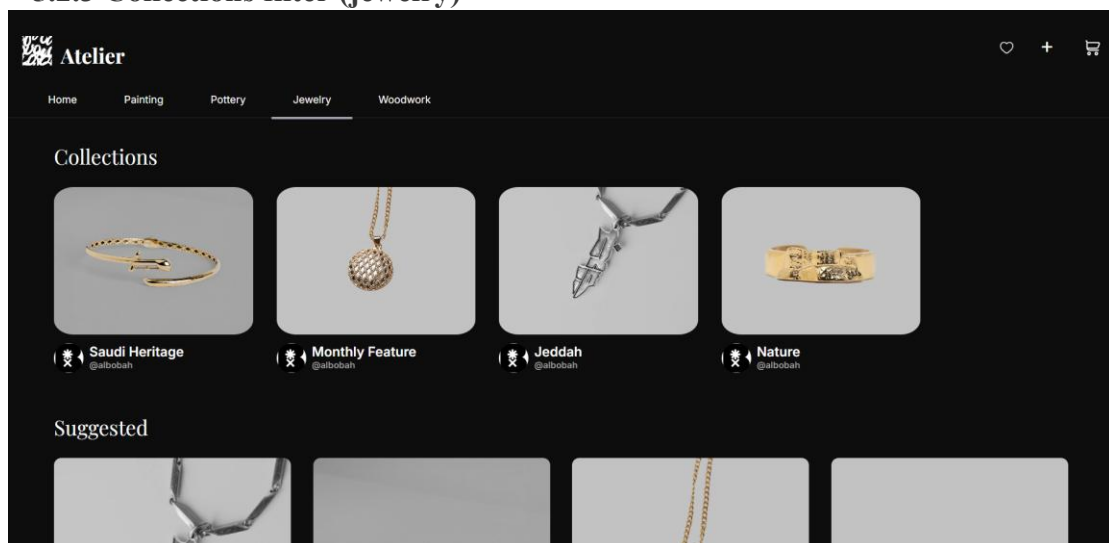
5.2.1 Collections filter (painting)



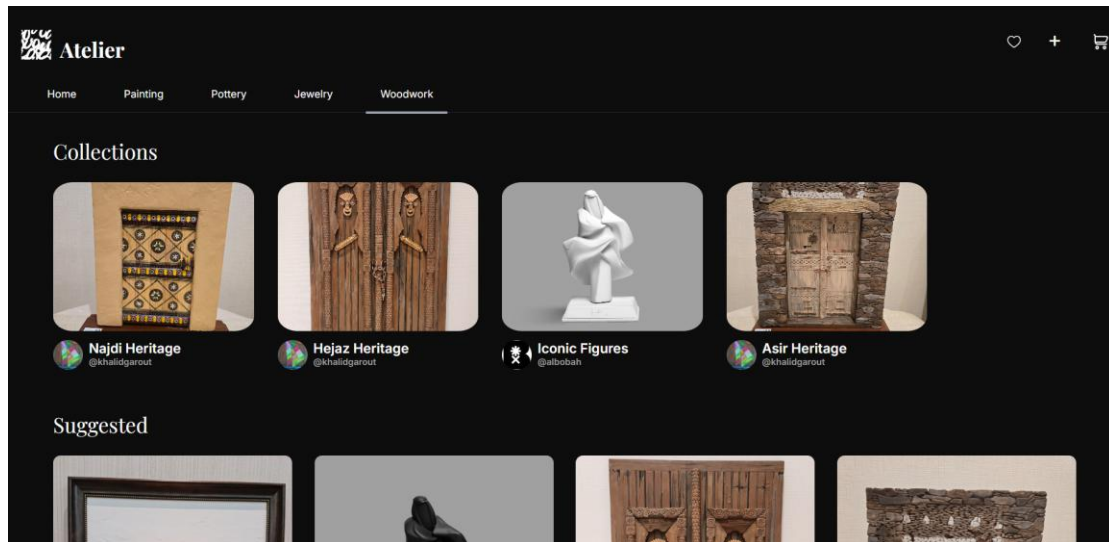
5.2.2 Collections filter (pottery)



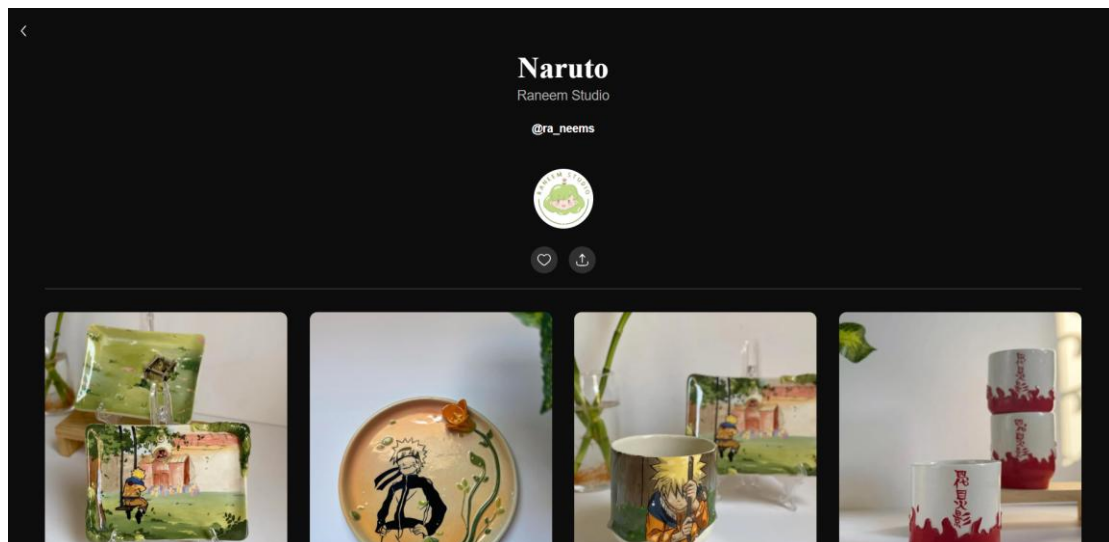
5.2.3 Collections filter (jewelry)



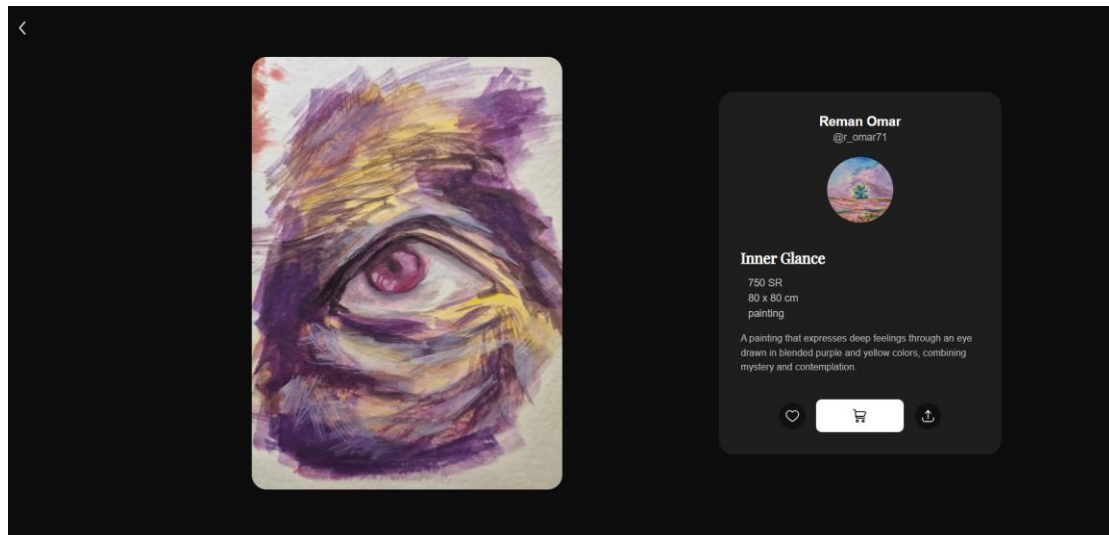
5.2.4 Collections filter (woodwork)



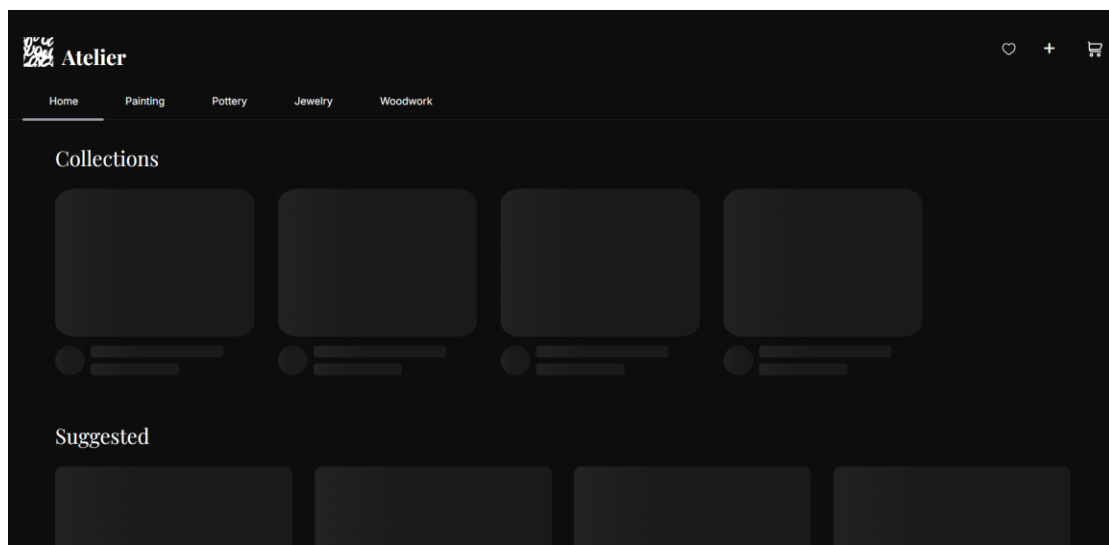
5.3 Collection Page



5.3 Product details Page

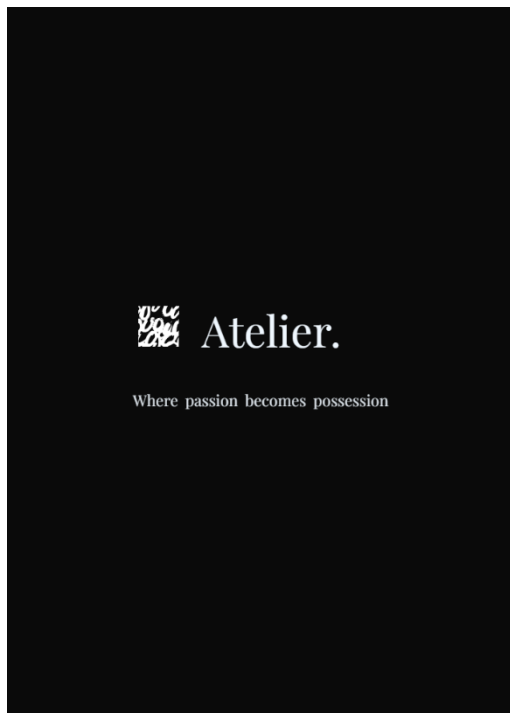


5.4 Loading skeleton

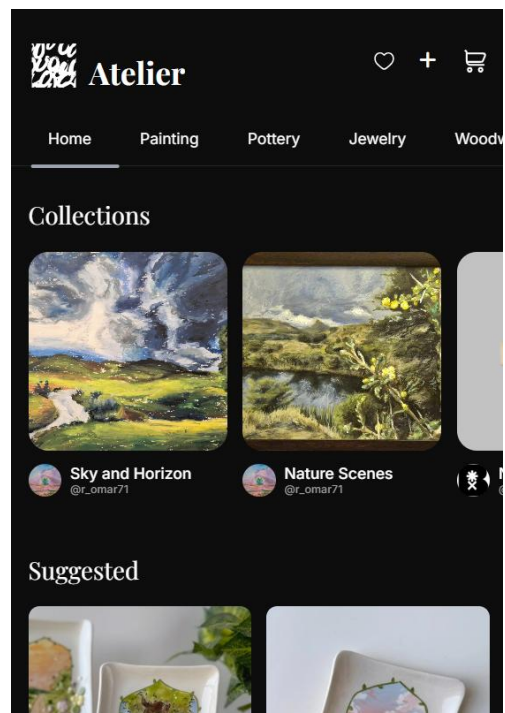


6 Implementation in Mobile (UI):

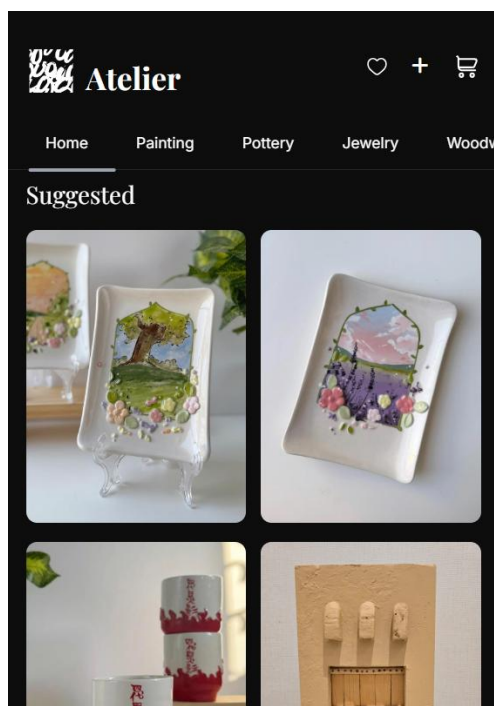
6.1 Splash screen



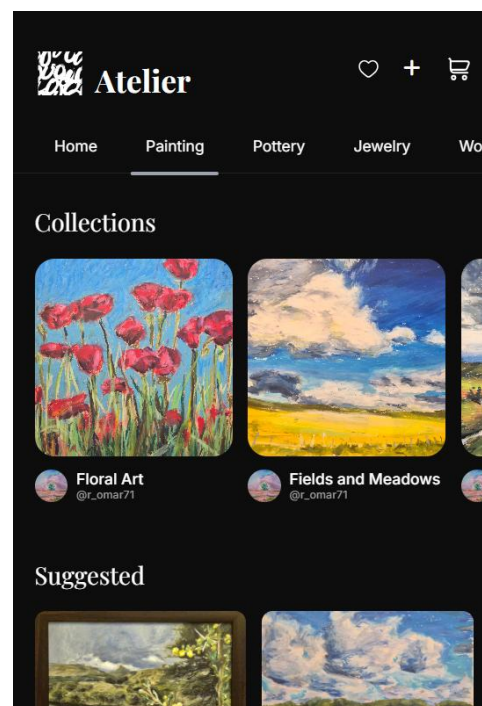
6.2 Home Page(collections)



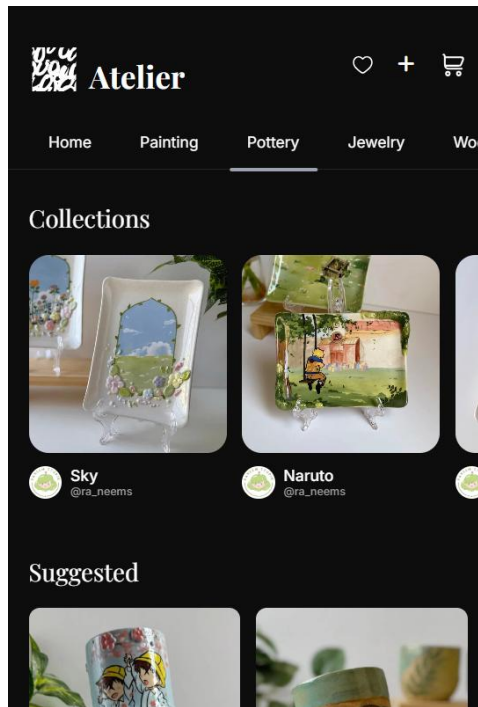
6.2 Home Page(suggestions)



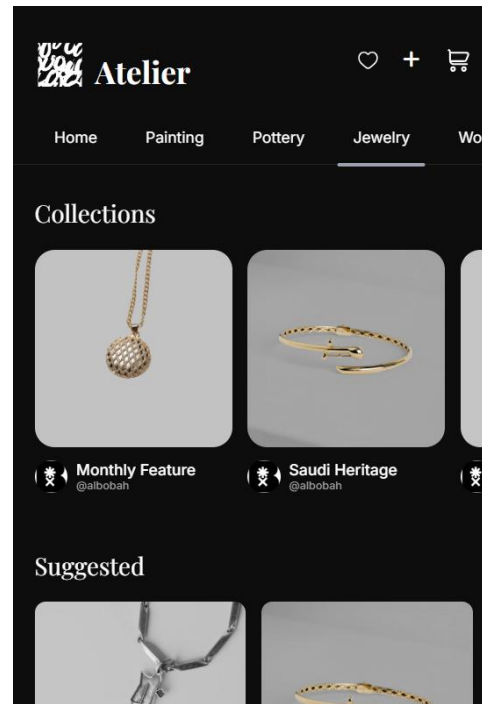
6.2.1 Collections filter (painting)



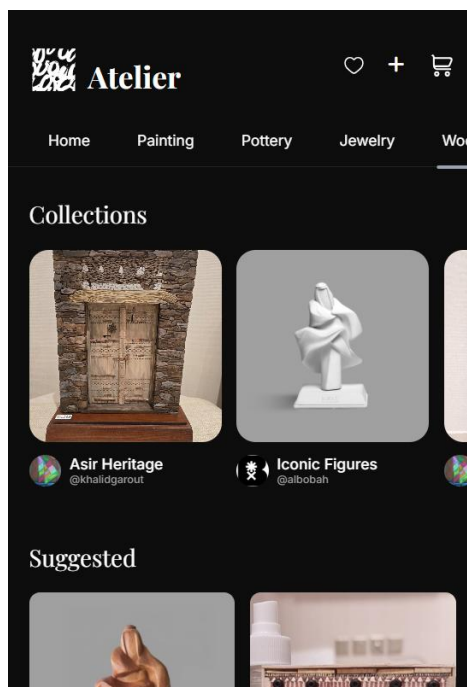
6.2.2 Collections filter (pottery)



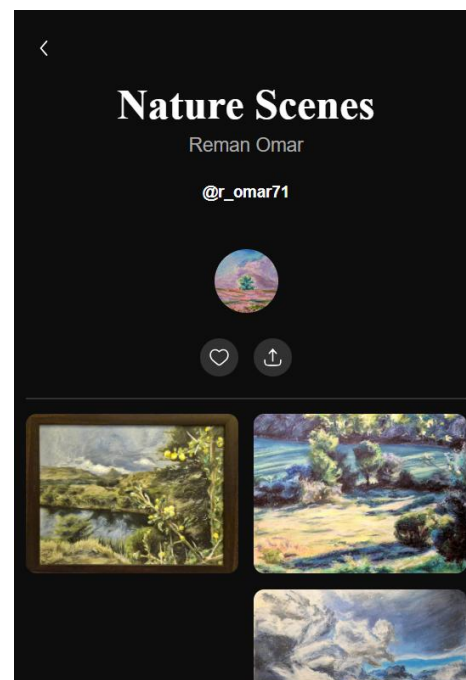
6.2.3 Collections filter (jewelry)



6.2.4 Collections filter (woodwork)



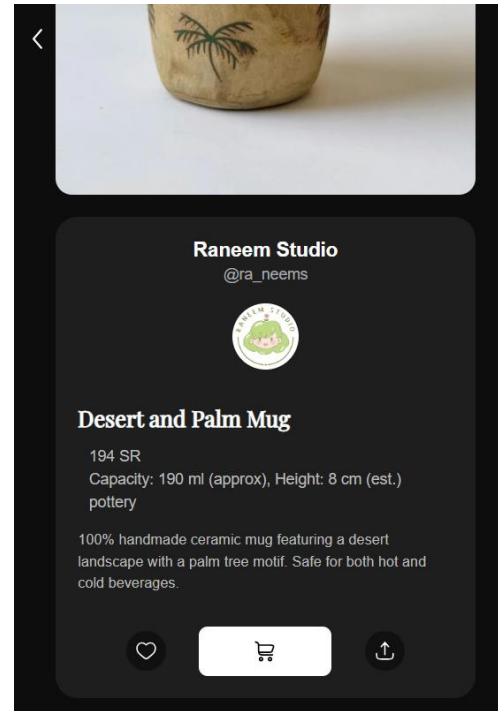
6.3 Collection Page



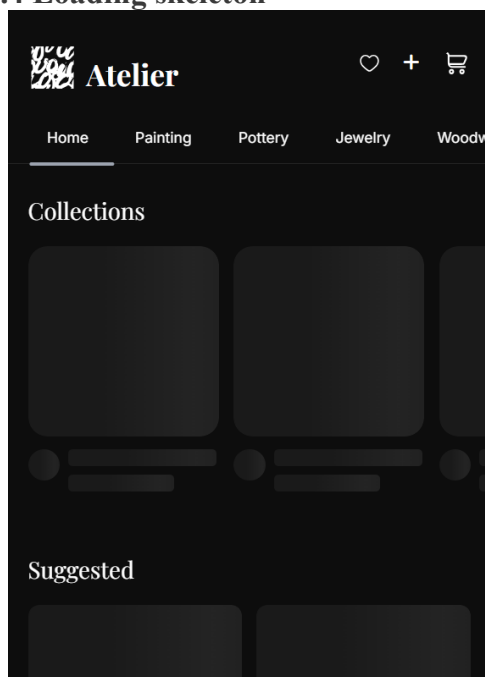
6.3.1 Product details Page



6.3.2 Product details Page



6.4 Loading skeleton



8 Test Cases

Test Case Name: Sprint 1-Browse main feed

Test Case Id: Atelier- Browse main feed

Test Case No.	Test Case Description	Expected Results	Outcome (Pass/Fail) Other (Comments))
TC001	User reloads page	Different products are displayed	Pass
TC002	User opens website from different devices	The layout adjusts accordingly	Pass
TC003	User change window size	The layout adjusts accordingly	Pass
TC004	User clicks on a category	Feed filters collection and products by category	Pass
TC005	User opens the website	Splash screen displays	Pass
TC006	User hovers over a specific product	The display scale of the product increases	Pass
TC007	User hovers over a specific collection	Collection thumbnail changes to different image	Pass
TC008	User clicks on a specific product	Navigates to the product detail page	Pass
TC009	User clicks on a specific collection	Navigates to the collection page	Pass

Test Case Name: Sprint 1-Browse collections

Test Case Id: Atelier- Browse collections

Test Case No.	Test Case Description	Expected Results	Outcome (Pass/Fail) Other (Comments))
TC001	User navigates to collection page	All collection data is displayed	Pass
TC002	User open website from different devices	The layout adjusts accordingly	Pass
TC003	User change window size	The layout adjusts accordingly	Pass
TC004	User hovers over a specific product	The display scale of the product increases	Pass
TC005	User clicks on a specific product	Navigates to the product detail page	Pass
TC006	User clicks on the back arrow	Navigates back to home page	Pass
TC007	User hovers over artist profile picture	The display scale of the picture will increase	Pass

Test Case Name: Sprint 1-View product details

Test Case Id: Atelier- View product details

Test Case No.	Test Case Description	Expected Results	Outcome (Pass/Fail) Other (Comments)
TC001	User navigates to product page	All product data is displayed	Pass
TC002	User opens website from different devices	The layout adjusts accordingly	Pass
TC003	User change window size	The layout adjusts accordingly	Pass
TC004	User hovers over artist profile picture	The display scale of the picture will increase	Pass
TC005	User clicks on the back arrow	Navigates back to home page	Pass

Sprint 2

1 Backlog (Name of functional requirement)

1.4 Components Name:

- Authentication
- Artisan Management
- Commission Management
- Wishlist

1.5 Functional Requirements:

FR-1: The system shall allow users to register and log in to their accounts

FR-9: The system shall allow users to save products to a personal wishlist.

FR-10: The system shall allow users to navigate to an artisan's full profile by selecting the artisan's username.

FR-11: The system shall display an artisan's profile page containing:

- The artisan's profile picture, and name.
- A bio "About the Artist"
- Links to the artisan's social media
- A "Request Commission" button
- A gallery of the artisan's previous works.

FR-12: The system shall provide filters for viewing artisan's work gallery "All Works" and "Available Works."

FR-13: The system shall provide a form for users to submit custom commission requests to an artisan, with the following fields: product type, dimensions, and a detailed description and attached media.

FR-14: The system shall automatically send an email alert to the artisan upon receiving a commission request

1.6 Non-Functional Requirements:

NFR-2: The website should be useable on both mobile phones and desktop computers

NFR-3: User data, product information, and orders must be saved correctly and not be lost after a page refreshing

NFR-4: The system shall maintain full functionality on Chrome, Firefox, Safari, and Edge.

NFR-7: The system data must be stored securely in the database

NFR-8 :Product images shall be optimized to minimize load time of 5 second

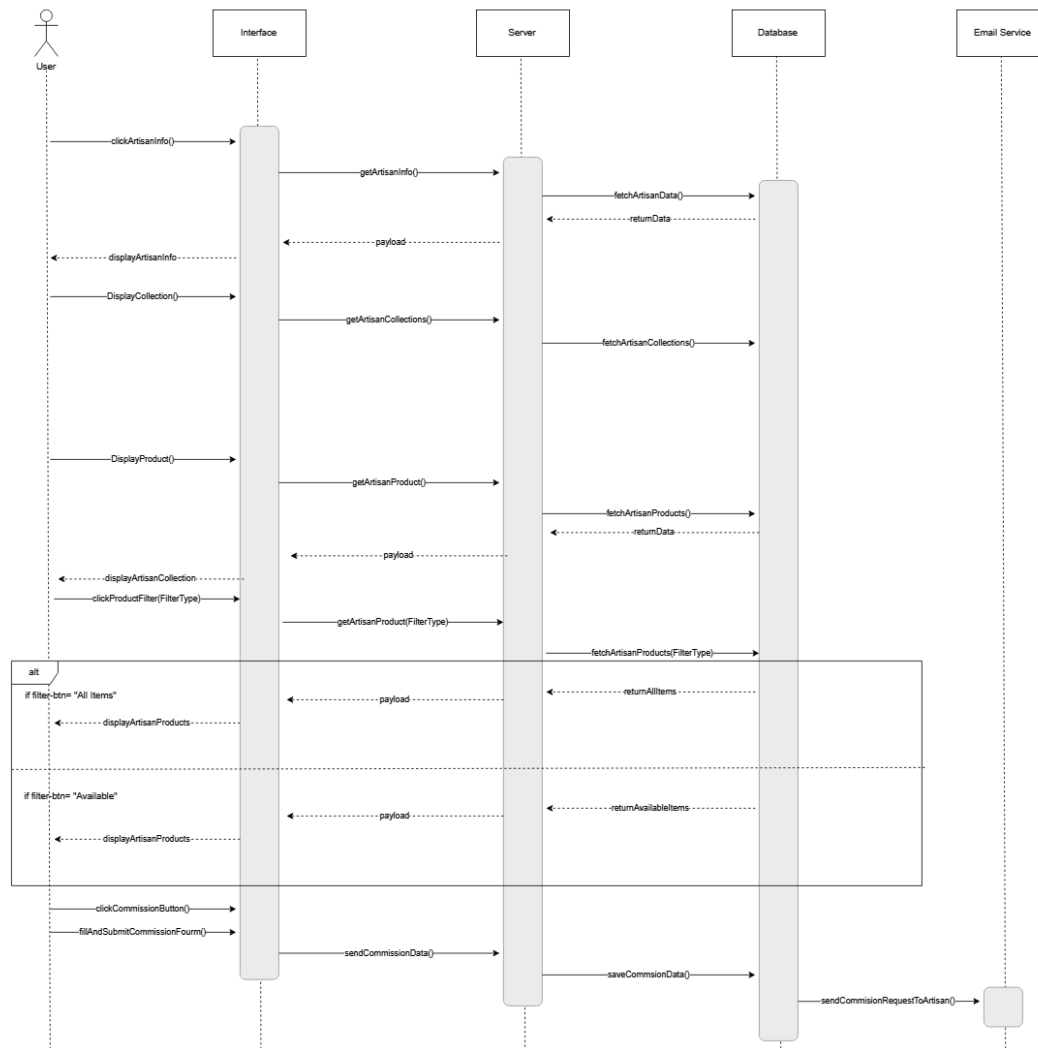
2 Diagrams

2.1 Use case:

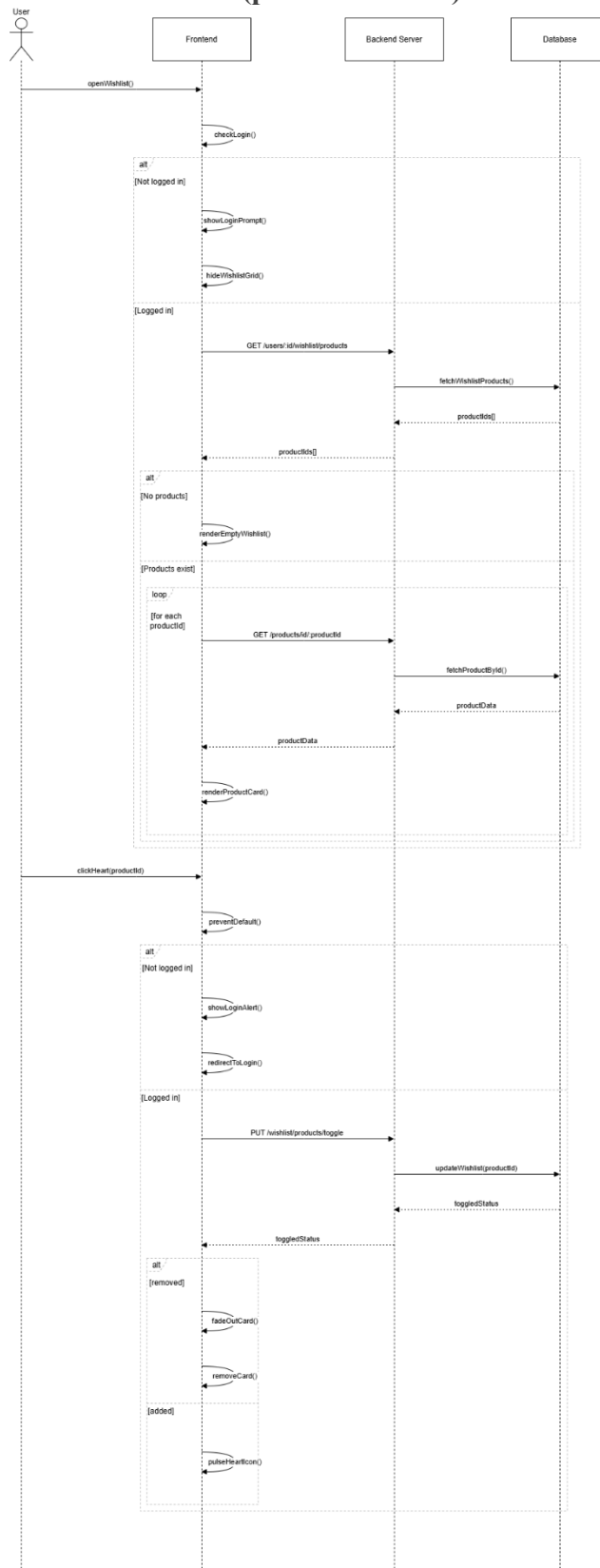


2.2 Sequence Diagram

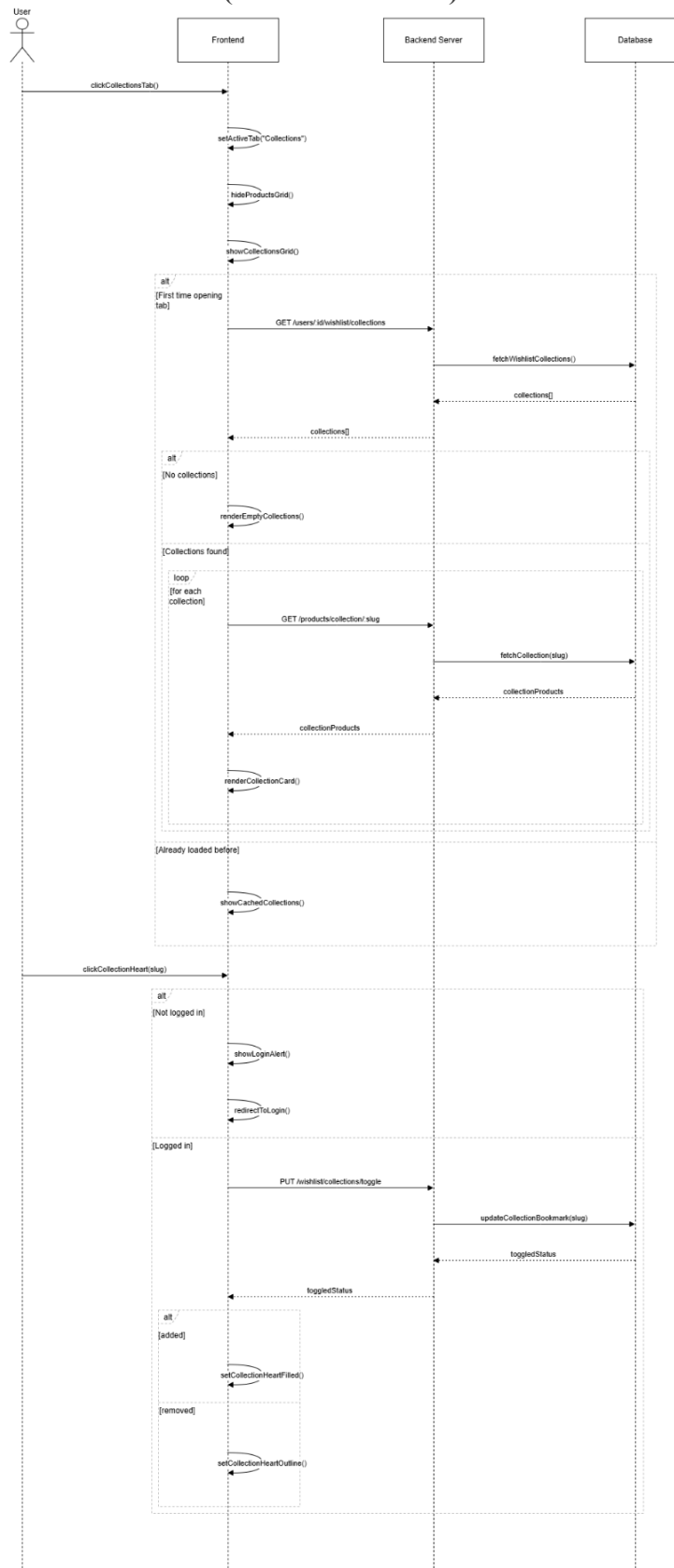
2.2.1 Profile Page



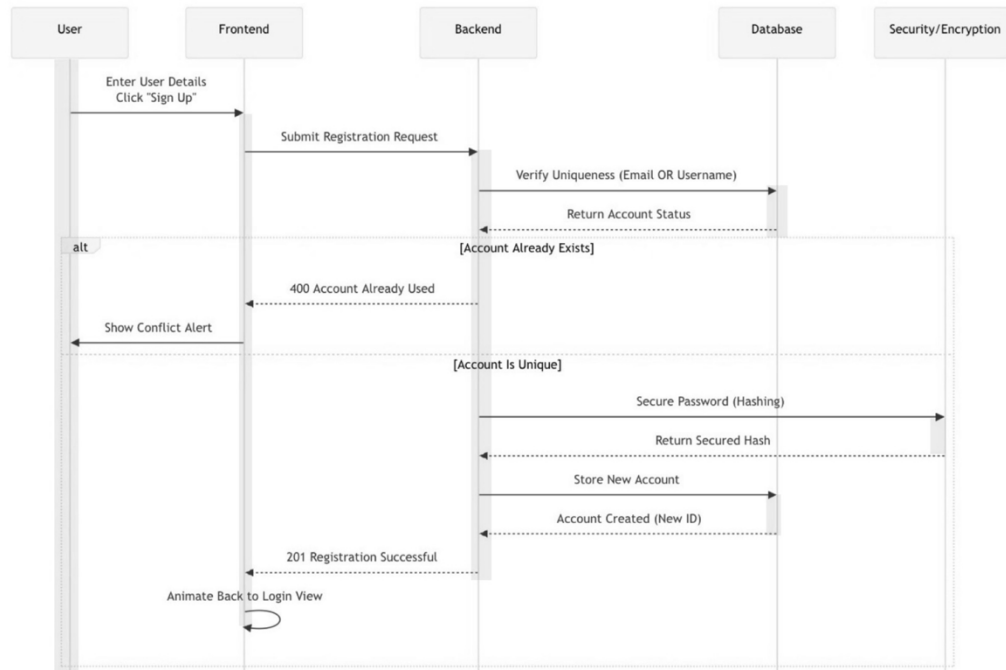
2.2.2 Wishlist: (product section)



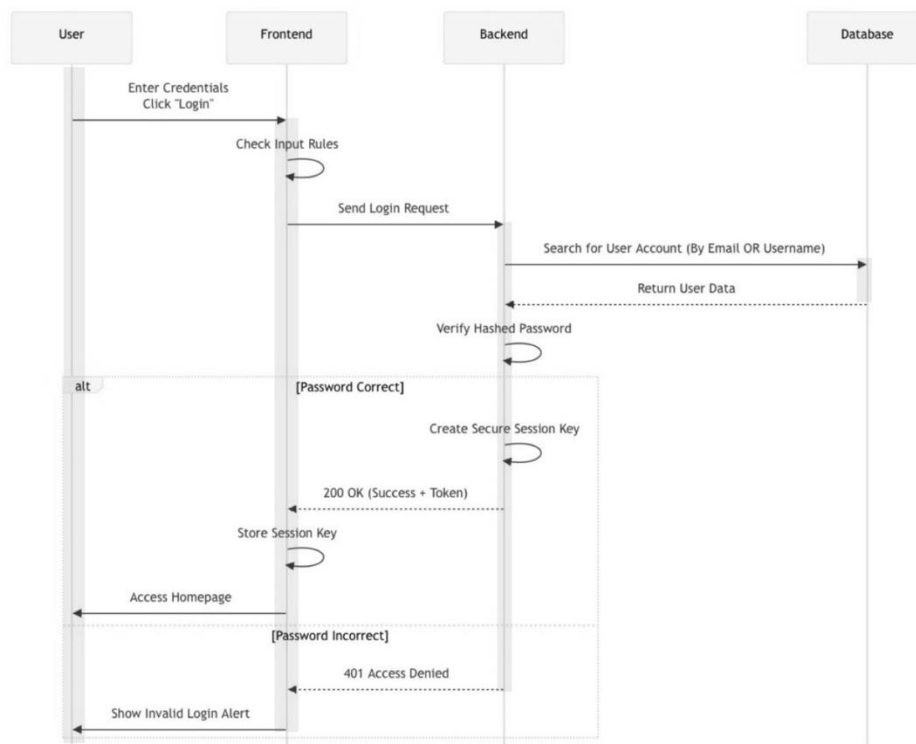
2.2.3 Wishlist: (collection section)



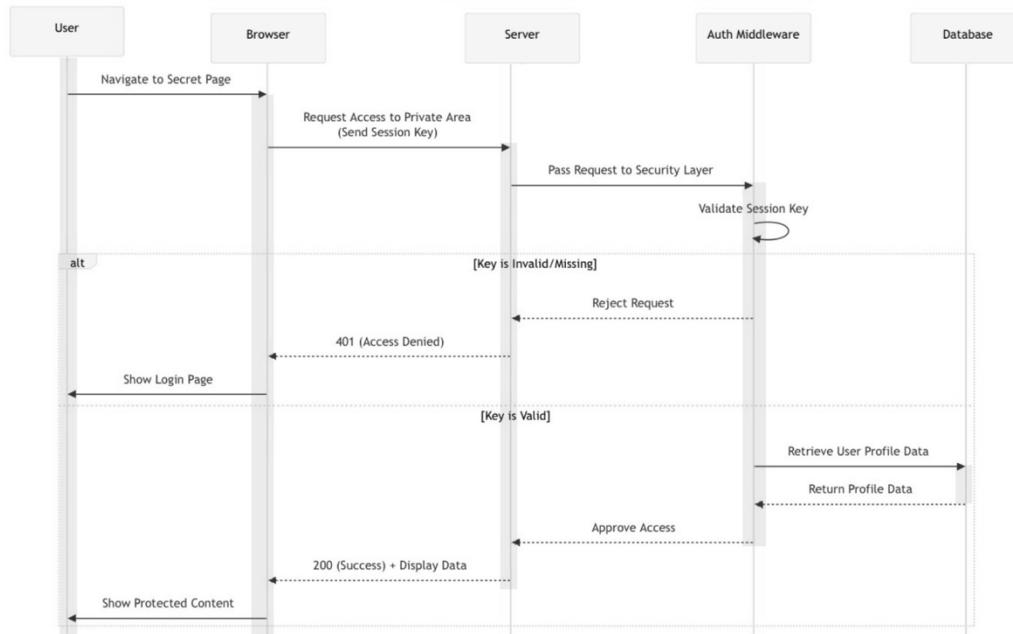
2.2.4 Sign Up:



2.2.5 Log In:



2.2.6 User Authorization:



3 Stories:

3.1 Login

Component Name: Authentication

Story Name: User Login

Story Sequence No: 01

Story Short Description: User logs into the system

Story Long Description: The user inputs email or username and password in order to authenticate and access their accounts.

3.2 Sign Up

Component Name: Authentication

Story Name: User Sign up

Story Sequence No: 02

Story Short Description: User creates new account

Story Long Description: A new user provides information (name , email, password and username) that are required to create a new account, allowing them to login and use full platform functionality.

3.3 View Artisan Profile

Component Name: Artisan Management

Story Name: View Artisan Profile

Story Sequence No: 03

Story Short Description: User views artisan details

Story Long Description: The user opens an artisan's profile to view their biography, artworks, social links and general information to discover more about the artisan.

3.4 Request Commission

Component Name: Commission Management

Story Name: Request Commission

Story Sequence No: 04

Story Short Description: User request custom artwork.

Story Long Description: The user submits a commission forum to a specific artisan by filling details allowing artisans to review and respond to clients.

3.5 View Wishlist

Component Name: Wishlist

Story Name: View Wishlist

Story Sequence No: 05

Story Short Description: User views saved products

Story Long Description: The user opens the wishlist page to view all the products they have already saved for later consideration or purchase.

3.6 Edit Wishlist

Component Name: Wishlist

Story Name: Add/Remove Wishlist Items

Story Sequence No: 06

Story Short Description: User manages Wishlist items.

Story Long Description: The user adds products to their Wishlist or removes them as they wish, allowing them to keep track of the items they are interested in.

4 Meetings:

Sprint Meeting(s)

Project Name: Atelier

Project Members:

Remas Alsulami - Marwa Hadaddi - Reman Alamoudi - Ghaida Almehmadi - Shumokh Alsharif

Sprint #2 *Stand up Meeting* [2 November 2025]

Sprint Duration: ...

Scrum Master: Shumokh Alsharif

Client: Malak Alharbi

Pair Programmers:

Remas Alsulami - Marwa Hadaddi - Reman Alamoudi - Ghaida Almehmadi - Shumokh Alsharif

Stories:

Components Name:

- Login/Signup page
- Artisan profile
- Wishlist page
- Commission form

Follow-up meeting questions:

4. What has been completed since the last meeting?
5. What are you going to be working on next?
6. Do you have any issues/impediments?

Scrum's Master comments based on the above questions:

- 4- Sprint tasks are distributed among team members
- 5- Designing the UI pages & start on the new server endpoint
- 6- So far none

Sprint Meeting(s)

Project Name: Atelier

Project Members:

Remas Alsulami - Marwa Hadaddi - Reman Alamoudi - Ghaida Almehmadi - Shumokh Alsharif

Sprint #2 *Progress Check In Meeting* [9 November 2025]

Sprint Duration: ...

Scrum Master: Shumokh Alsharif

Client: Malak Alharbi

Pair Programmers:

Remas Alsulami - Marwa Hadaddi - Reman Alamoudi - Ghaida Almehmadi - Shumokh Alsharif

Stories:

Components Name:

- Login/Signup page
- Artisan profile
- Wishlist page
- Commission form

Follow-up meeting questions:

7. What has been completed since the last meeting?
8. What are you going to be working on next?
9. Do you have any issues/impediments?

Scrum's Master comments based on the above questions:

- 7- Frontend pages and UI are fully built, new schema are added, mail service configured
- 8- Build the endpoints for the new pages, create a dynamic email template, link frontend with backend
- 9- No far none

Sprint Meeting(s)

Project Name: Atelier

Project Members:

Remas Alsulami - Marwa Hadaddi - Reman Alamoudi - Ghaida Almeahmadi - Shumokh Alsharif

Sprint #2 Progress Check In Meeting [11 November 2025]

Sprint Duration: ...

Scrum Master: Shumokh Alsharif

Client: Malak Alharbi

Pair Programmers:

Remas Alsulami - Marwa Hadaddi - Reman Alamoudi - Ghaida Almeahmadi - Shumokh Alsharif

Stories:

Components Name:

- Login/Signup page
- Artisan profile
- Wishlist page
- Commission form

Follow-up meeting questions:

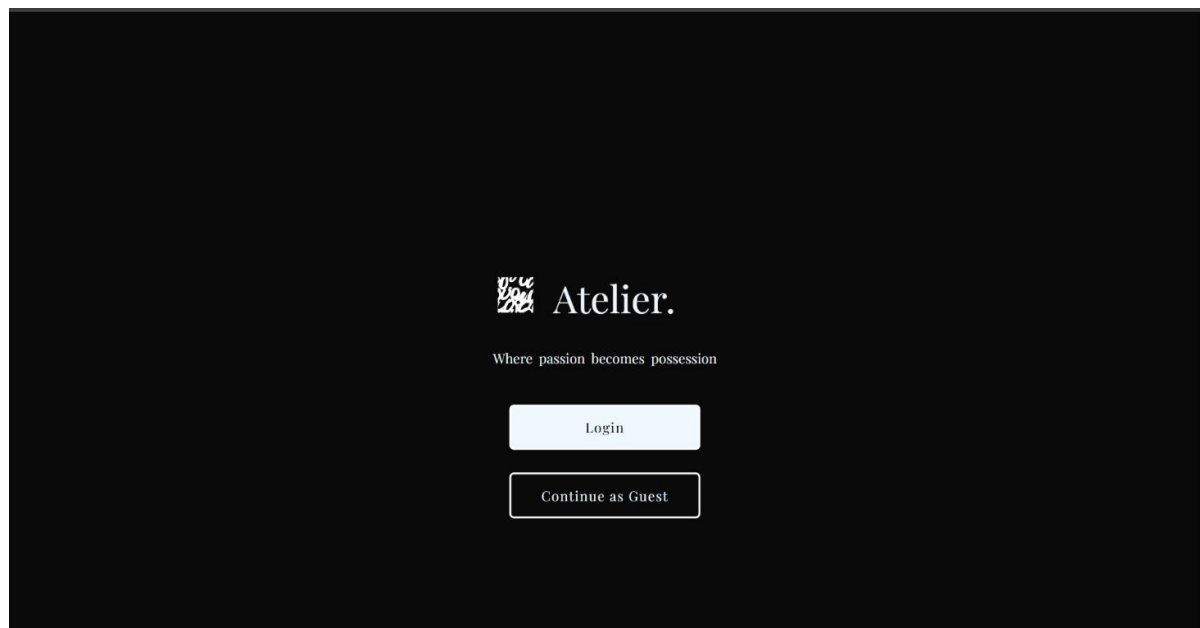
10. What has been completed since the last meeting?
11. What are you going to be working on next?
12. Do you have any issues/impediments?

Scrum's Master comments based on the above questions:

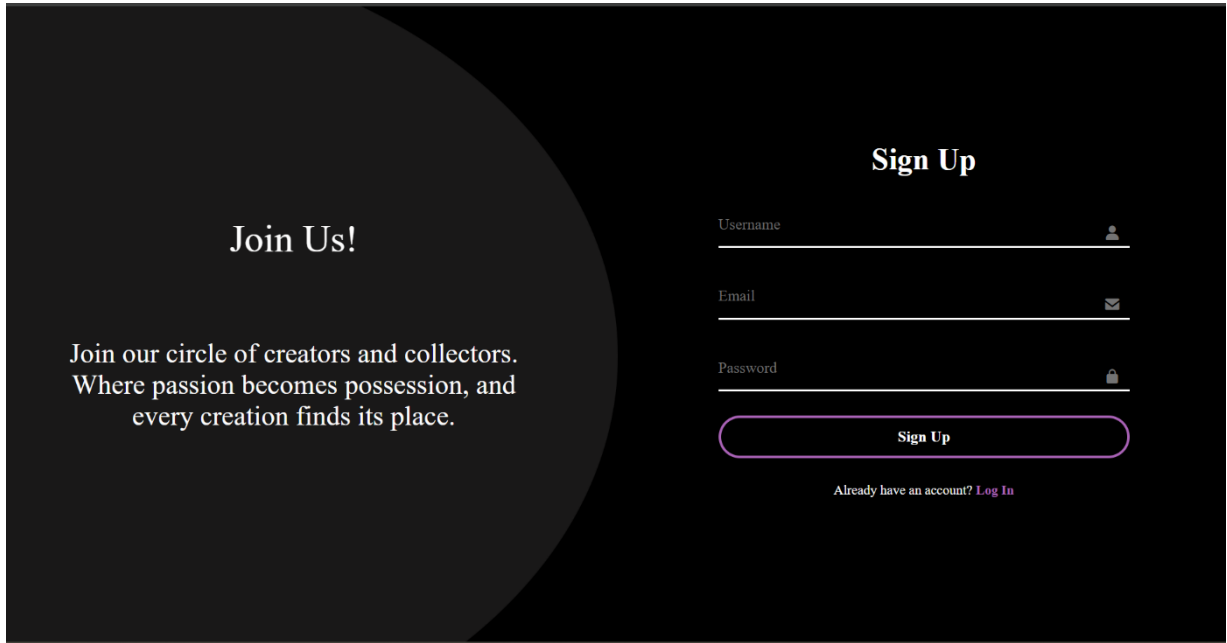
- 10- All new endpoints are added to the server, the frontend pages and complete, started configuring nodemailer
- 11- Link the commission form to the backend, adding navigation between pages, improve some of the endpoint's responses, link the login logic with the local storage
- 12- The current server host blocks SMTP protocols

5 Implementation in Web browser (UI):

5.1 Welcome Page



5.2 Sign in Page



The Sign Up page features a dark background with a large, light gray circular graphic on the left. The text 'Join Us!' is prominently displayed in white. Below it, a paragraph reads: 'Join our circle of creators and collectors. Where passion becomes possession, and every creation finds its place.' On the right side, the 'Sign Up' form is presented with three input fields: 'Username' (with a person icon), 'Email' (with an envelope icon), and 'Password' (with a lock icon). A purple 'Sign Up' button is positioned below the fields. At the bottom, a link states 'Already have an account? [Log In](#)'.

Join Us!

Join our circle of creators and collectors.
Where passion becomes possession, and
every creation finds its place.

Sign Up

Username 

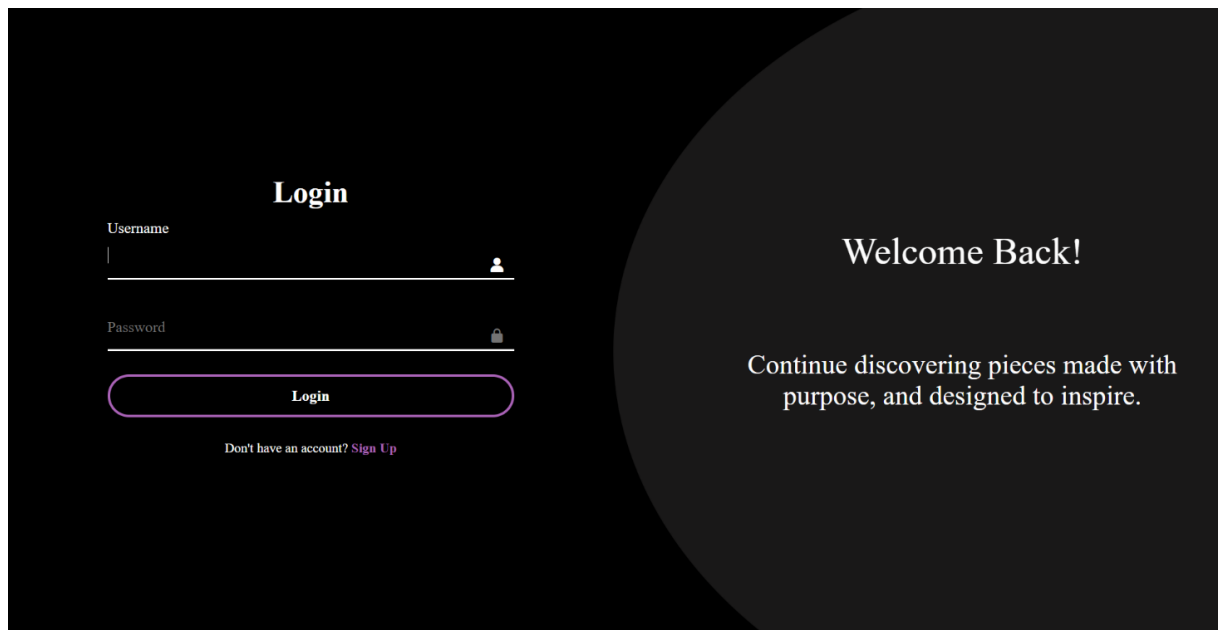
Email 

Password 

Sign Up

Already have an account? [Log In](#)

5.3 Login Page



The Login page features a dark background with a large, light gray circular graphic on the right. The text 'Welcome Back!' is prominently displayed in white. Below it, a paragraph reads: 'Continue discovering pieces made with purpose, and designed to inspire.' On the left side, the 'Login' form is presented with two input fields: 'Username' (with a person icon) and 'Password' (with a lock icon). A purple 'Login' button is positioned below the fields. At the bottom, a link states 'Don't have an account? [Sign Up](#)'.

Login

Username 

Password 

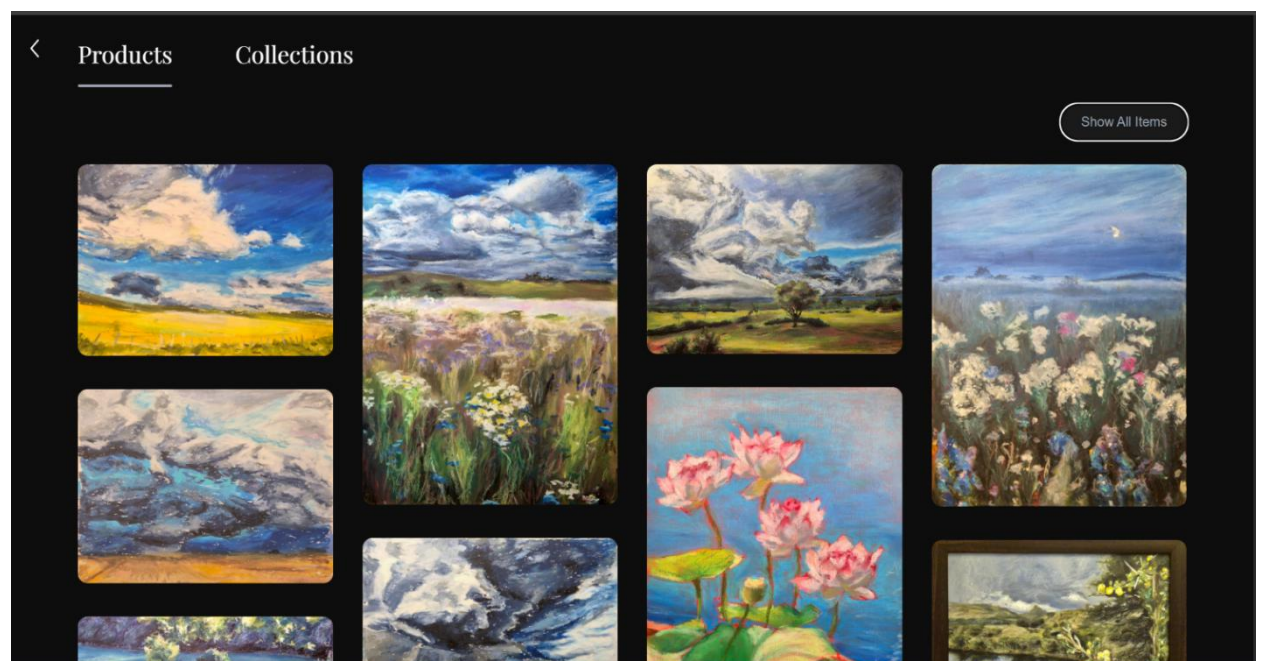
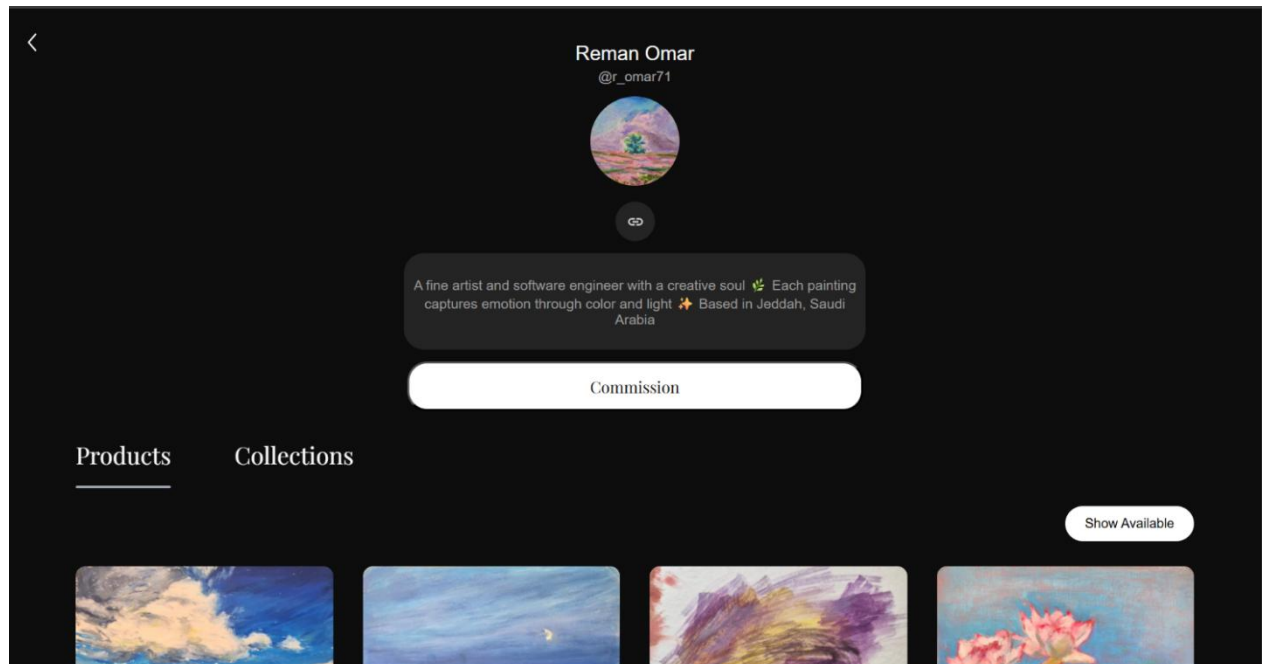
Login

Don't have an account? [Sign Up](#)

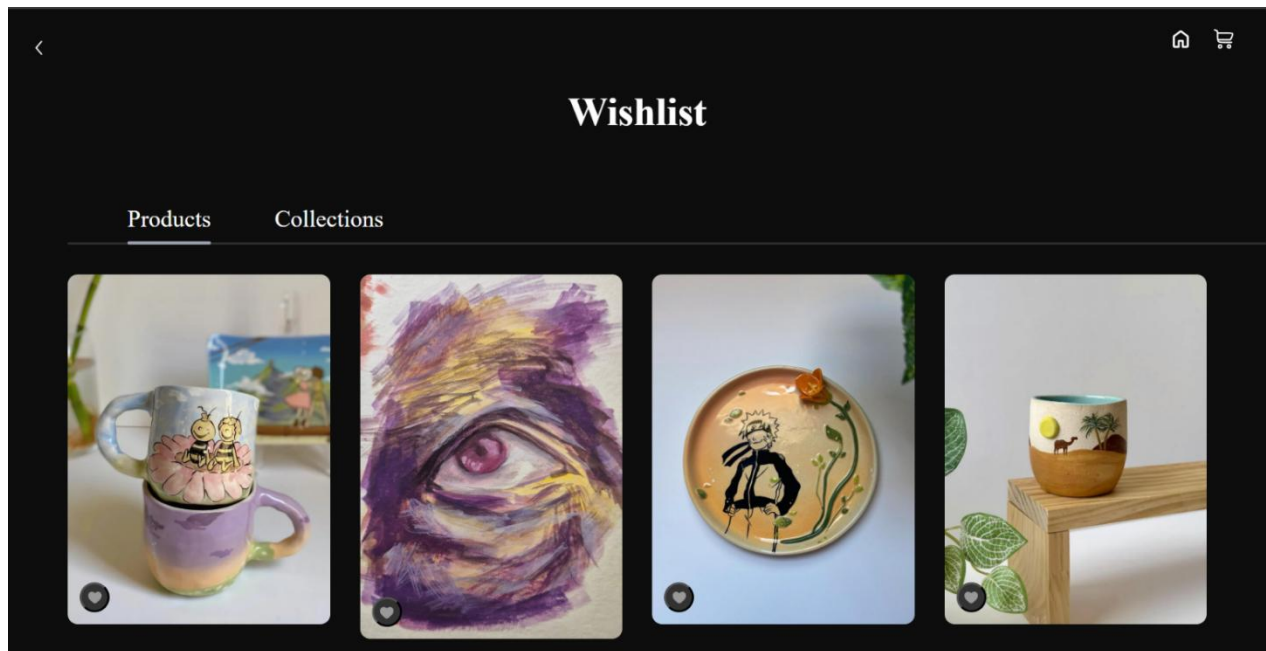
Welcome Back!

Continue discovering pieces made with
purpose, and designed to inspire.

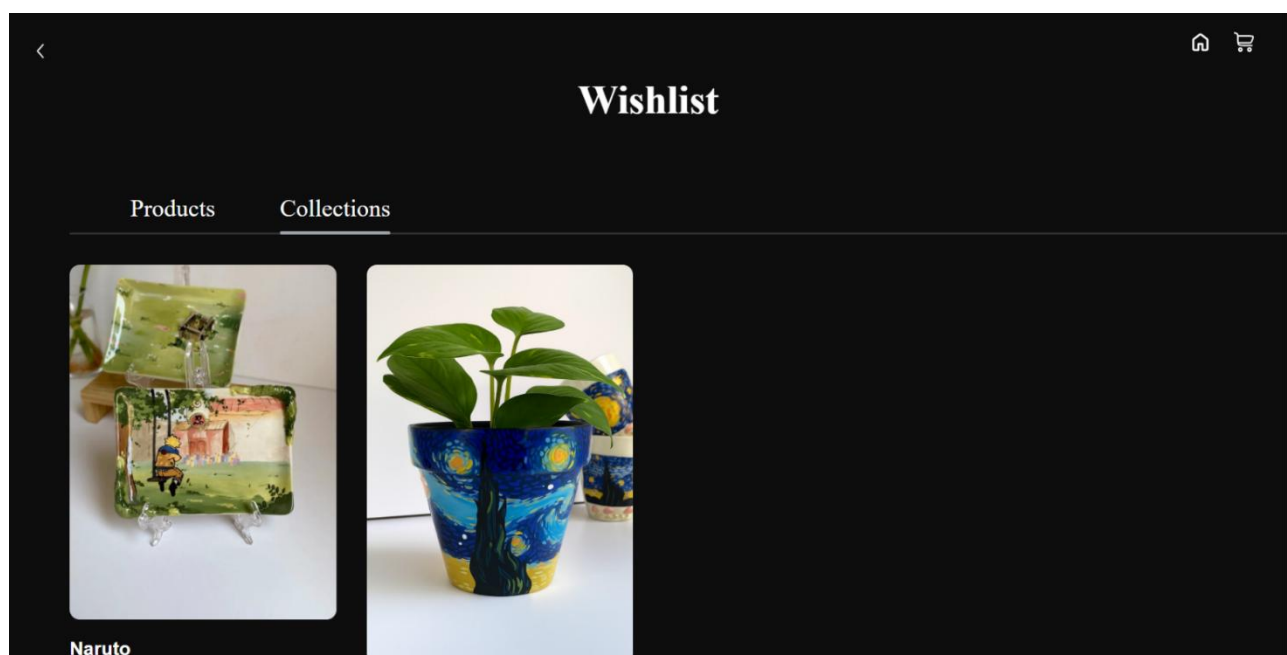
5.4 Profile Page



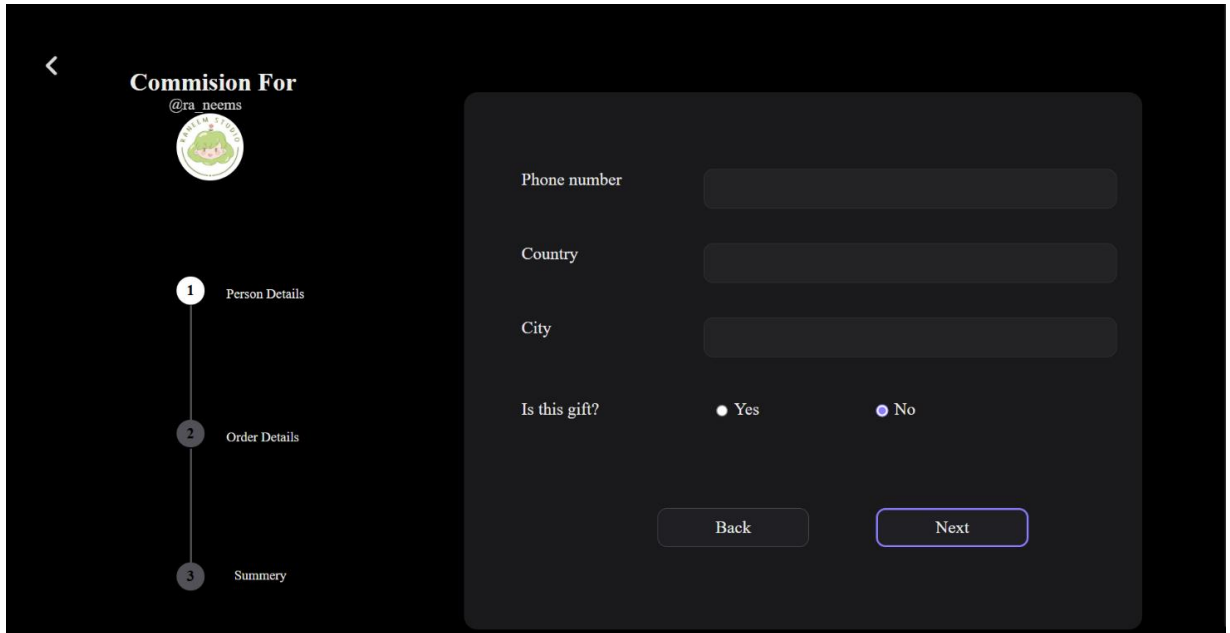
5.5 Wishlist Page



5.5.1 Wishlist Page

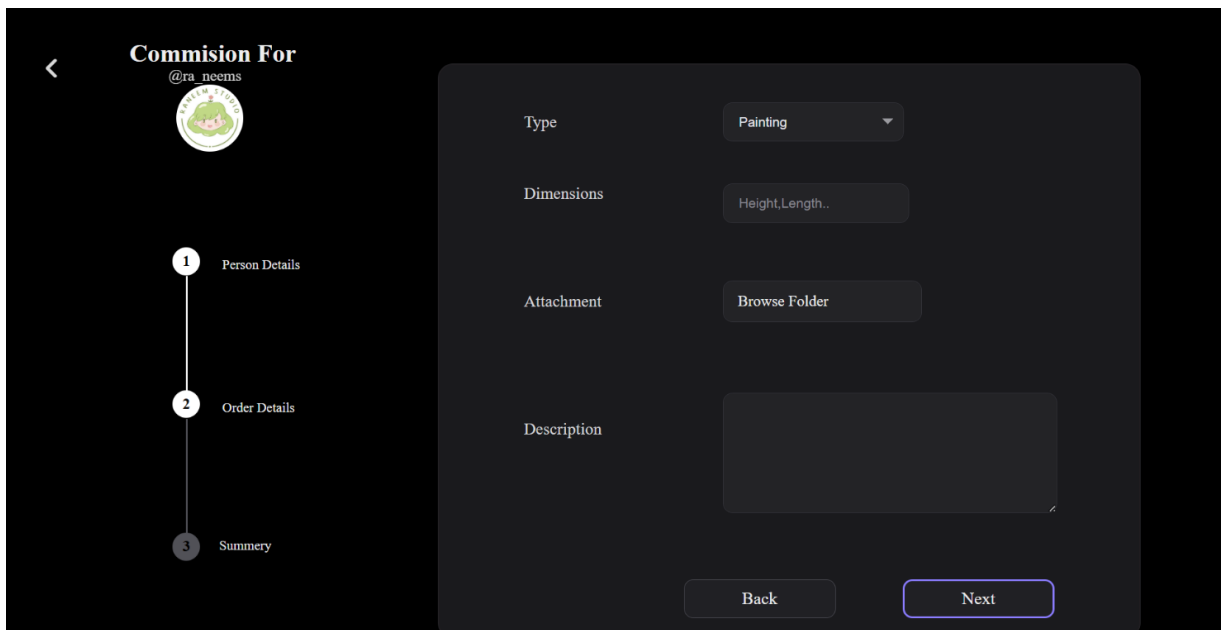


5.6 Commission Page



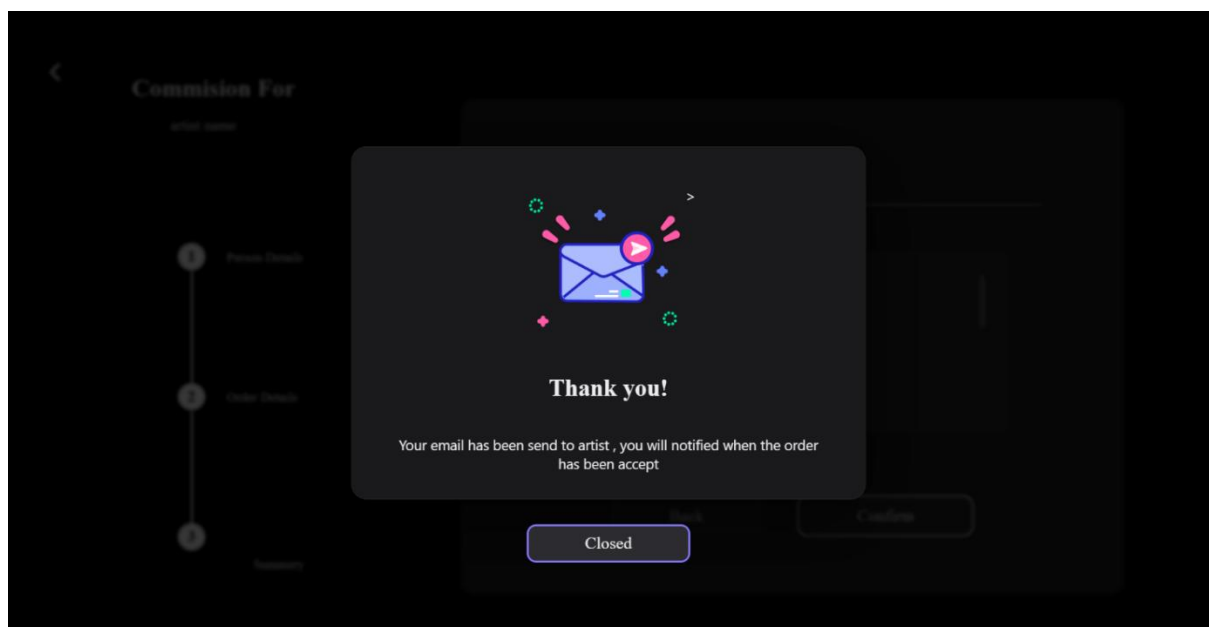
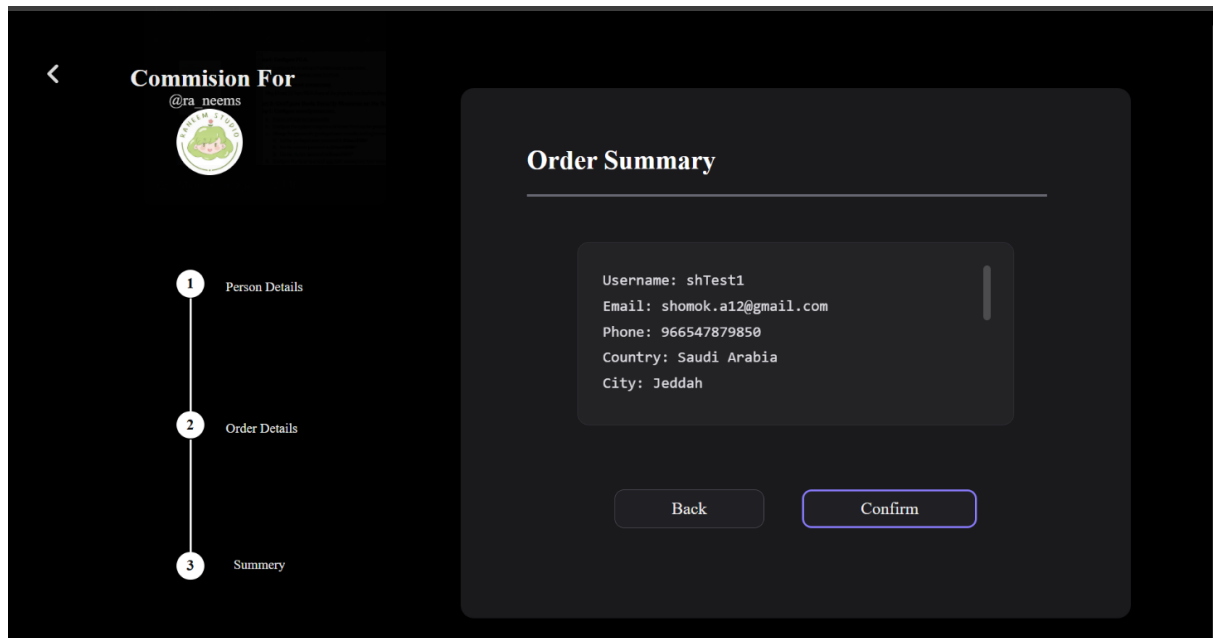
The screenshot shows a mobile application interface for a commission form. On the left, there is a vertical progress bar with three steps: 1. Person Details (highlighted), 2. Order Details, and 3. Summary. The main content area is a dark gray form with the following fields: Phone number, Country, City, and Is this gift? (with radio buttons for Yes and No, where No is selected). At the bottom of the form are two buttons: Back and Next.

5.6.1 Commission Page



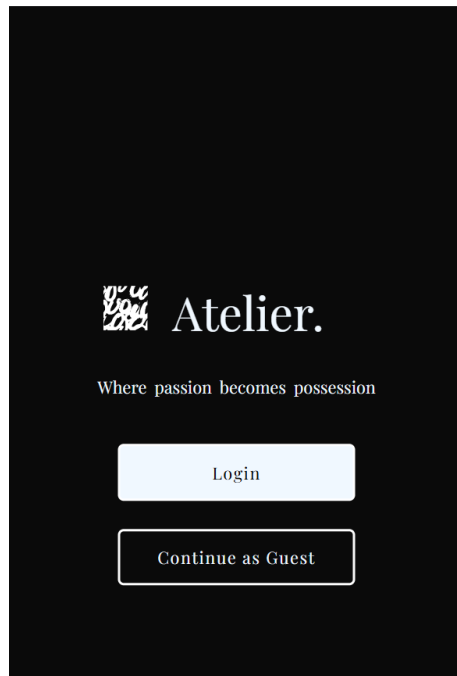
The screenshot shows the same mobile application interface, but now on the 'Order Details' step (step 2). The progress bar highlights step 2. The form fields are: Type (a dropdown menu showing 'Painting'), Dimensions (a text input field with 'Height, Length..'), Attachment (a button labeled 'Browse Folder'), and Description (a large text area). At the bottom are Back and Next buttons.

5.6.2 Commission Page

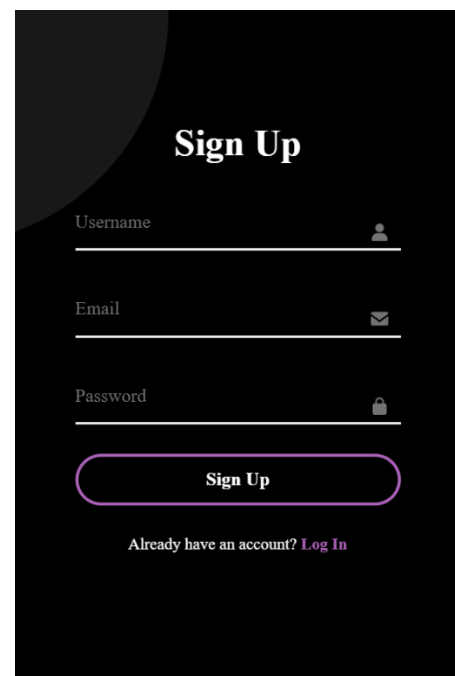


7 Implementation in Mobile (UI):

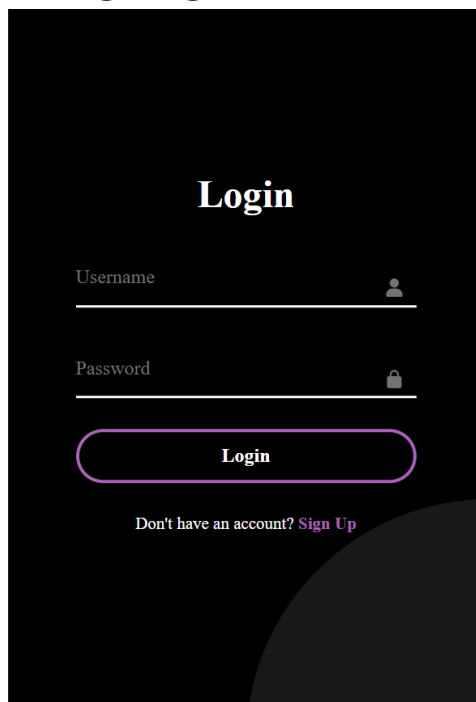
7.1 Welcome Page



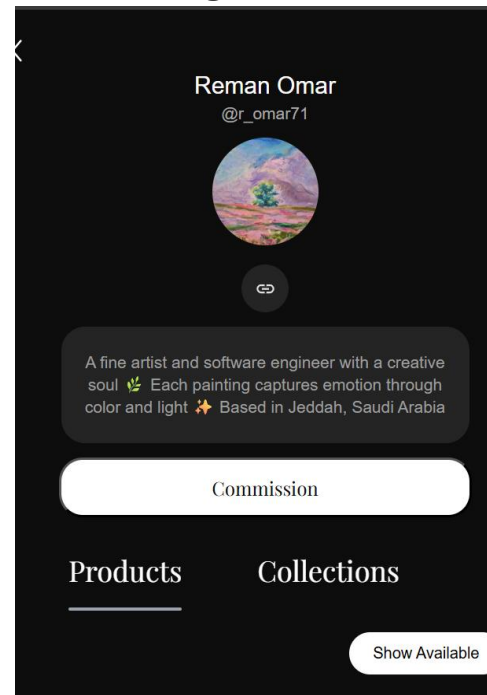
7.2 Sign in Page



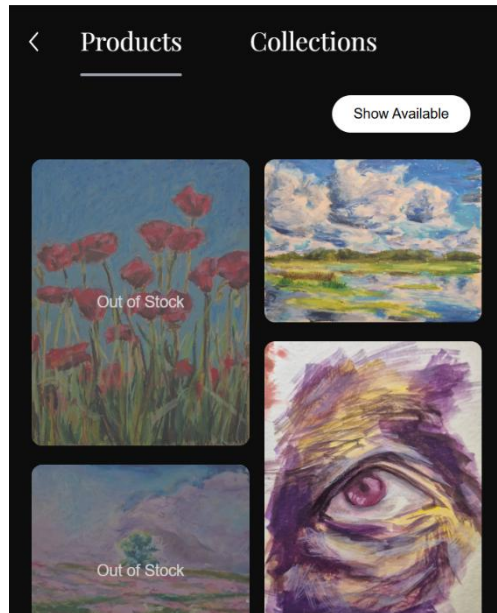
7.3 Login Page



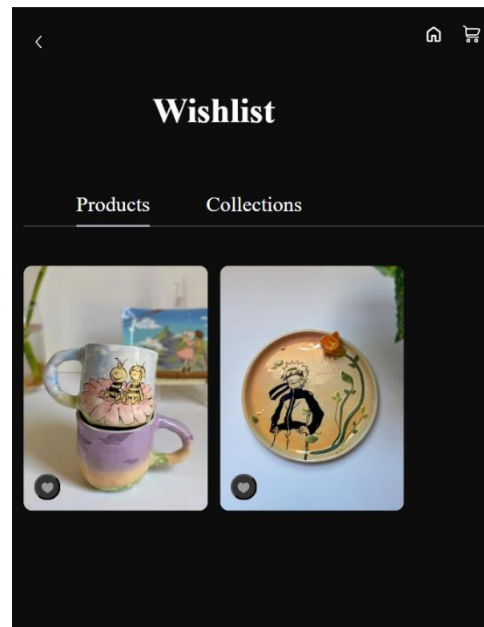
7.4 Profile Page



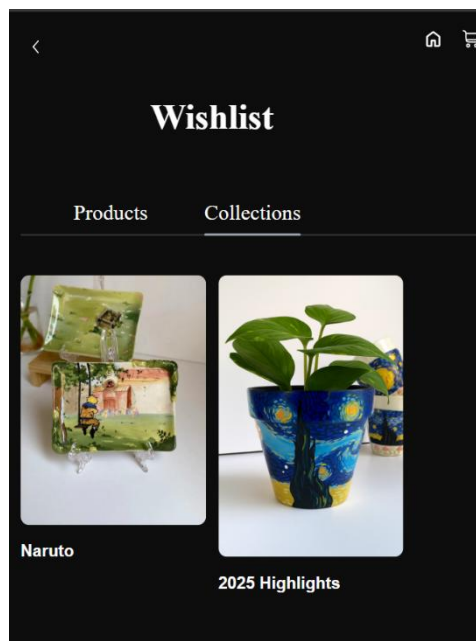
7.4.1 Profile Page



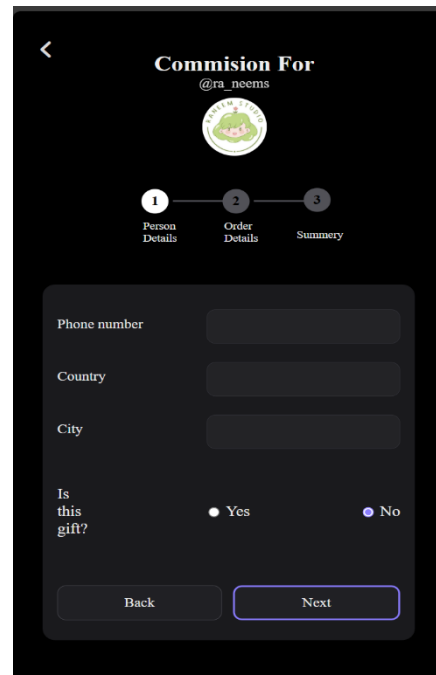
7.5 Wishlist Page



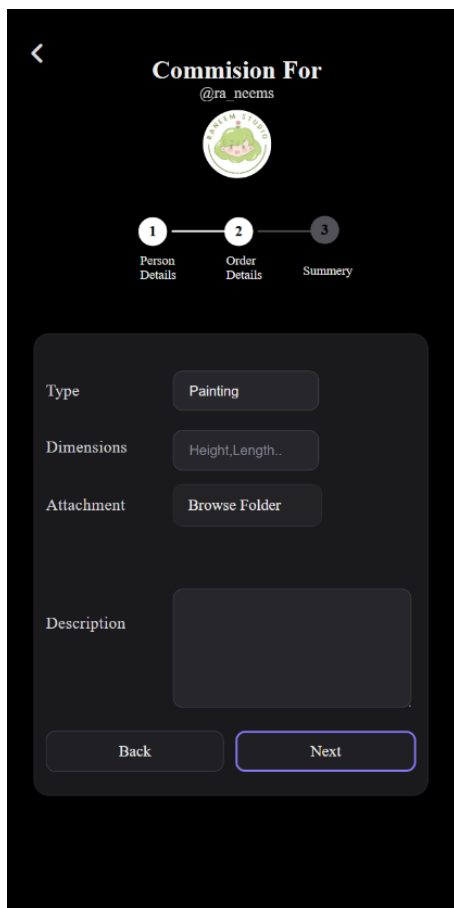
7.5.1 Wishlist Page



7.6 Commission Page



7.6.1 Commission Page



< Commission For
@ra_neems

1 2 3
Person Details Order Details Summary

Type Painting

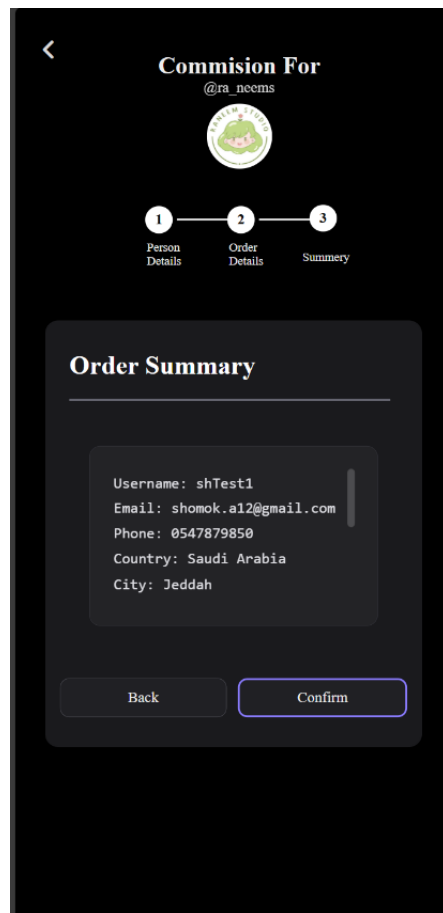
Dimensions Height, Length..

Attachment Browse Folder

Description

Back Next

7.6.2 Commission Page



< Commission For
@ra_neems

1 2 3
Person Details Order Details Summary

Order Summary

Username: shTest1
Email: shomok.a12@gmail.com
Phone: 0547879850
Country: Saudi Arabia
City: Jeddah

Back Confirm

8 Test Cases

Test Case Name: Sprint 2- Log In

Test Case Id: Atelier – Log In

Test Case No.	Test Case Description	Expected Results	Outcome (Pass/Fail) Other (Comments))
TC001	User successfully logs in with valid Username and Password.	User is redirected to the main dashboard/feed.	Pass
TC002	User attempts to log in with an invalid Username but a valid Password.	An error message is displayed: "Invalid username or password."	Pass
TC003	User attempts to log in with a valid Username but an invalid Password.	An error message is displayed: "Invalid username or password."	Pass
TC004	User attempts to log in leaving the Username field empty.	A validation error/message prompts the user to enter the username.	Pass
TC005	User attempts to log in leaving the Password field empty.	A validation error/message prompts the user to enter the password.	Pass
TC006	User clicks the "Sign Up" or "Register Now" link on the Login page.	User is successfully navigated to the User Registration page.	Pass

Test Case Name: Sprint 2 - Registration

Test Case Id: Atelier – Registration

Test Case No.	Test Case Description	Expected Results	Outcome (Pass/Fail) Other (Comments)
TC001	User successfully registers with all valid and required information (Username, Email, Password).	User account is created, and the user is logged in/directed to a success page.	Pass
TC002	User attempts to register using an email address that is already in use.	An error message is displayed: "This email address is already in use."	Pass
TC003	User attempts to register with an invalid email format (e.g., missing @ or .com).	An immediate validation error is displayed, guiding the user to correct the email format.	Pass
TC004	User attempts to register while leaving one of the required fields (e.g., Username) empty.	A validation error/message prompts the user to fill in the missing required field.	Pass
TC005	After successful registration, the user receives a confirmation dialog and clicks continue.	User is successfully navigated to the Login page.	Pass

Test Case Name: Sprint 2- Wishlist page

Test Case Id: Atelier - Wishlist page

Test Case No.	Test Case Description	Expected Results	Outcome (Pass/Fail) Other (Comments))
TC001	Unregistered user opens the Wishlist page	Nothing is displayed and user will be prompted to log in first	Pass
TC002	Unregistered users try to add an item to the Wishlist	Product/collection won't be added and user will be prompted to log in first	Pass
TC003	Registered user clicks on the heart icon	The product/collection will be added to their Wishlist	Pass
TC004	Registered user opens the Wishlist page	All the products/collections on their Wishlist will be displayed	Pass
TC005	Registered user clicks on the heart icon for an item already in the Wishlist	Product/collection will be removed from the Wishlist	Pass
TC006	User change window size	The layout adjusts accordingly	Pass

Test Case Name: Sprint 2- commission

Test Case Id: Atelier - commission

Test Case No.	Test Case Description	Expected Results	Outcome (Pass/Fail) Other (Comments))
TC001	User clicks on the commission button	User goes to the commission form	Pass
TC002	User forgets a required field	The required field will vibrate, and the user won't continue until it is filled	Pass
TC003	User clicks on the next button	It takes them to the next part of the commission form	Pass
TC004	User clicks back	It takes them to the previous part of the form	Pass
TC005	User chooses to attach a file or a photo	The device file system is opened	Pass
TC006	User submits their commission	A confirmation message is displayed on screen	Pass
TC007	A new commission is received	Both the user and artisan get an email notification	Pass
TC008	User change window size	The layout adjusts accordingly	Pass

Test Case Name: Sprint 2- artist profile

Test Case Id: Atelier - artist profile

Test Case No.	Test Case Description	Expected Results	Outcome (Pass/Fail) Other (Comments))
TC001	User clicks on the artist's photo/username on the Home Page.	User is successfully navigated to the Artist Profile Page, displaying their information and works.	Pass
TC002	User clicks on the artist's photo/username on the Collection Page.	User is successfully navigated to the Artist Profile Page.	Pass
TC003	User clicks on the artist's photo/username on the Product Page.	User is successfully navigated to the Artist Profile Page.	Pass
TC004	User clicks on the artist's social media icon/link.	The artist's social media account opens (preferably in a new window/tab).	Pass
TC005	User hovers the mouse over an out-of-stock product.	The text "Out of Stock" appears, and the product image is overlaid with a slight gray transparency.	Pass
TC006	User clicks the "Availability" button.	The filter is applied, and only "Available" products are displayed.	Pass
TC007	User clicks the "Availability" button again.	The filter is removed, and "All products" (Available and Out of Stock) are displayed.	Pass
TC008	User clicks on any "product" on the artist's page.	User is navigated to the product's specific page (Product/Collection page).	Pass
TC009	User clicks the "Back" button from the product page.	User successfully returns to the Artist Profile Page.	Pass
TC010	User clicks on the "Collections" button/word.	All of the artist's collections are displayed with their names.	Pass

TC0١١	User clicks on the “Commission” button.	User is navigated to the artist’s specific Commission request page.	Pass
TC0١٢	The Artist Profile Page loads.	The artist’s name, profile picture, and biography (Bio) are all displayed correctly.	Pass

Sprint 3

1 Backlog (Name of functional requirement)

1.1 Components Name:

- Cart
- Checkout
- Web-AR

1.2 Functional Requirements:

FR-5: The system shall allow users to add products to a shopping cart.

FR-6: The system shall group products by their respective shop (artisan) in the cart

FR-8: The system shall calculate the total of the cart.

FR-9: The system shall provide a checkout process with the following payment methods: Apple Pay, debit card, credit card.

FR-17: The system shall implement a Web-based Augmented Reality (Web AR) feature to allow users to visualize products in their environment.

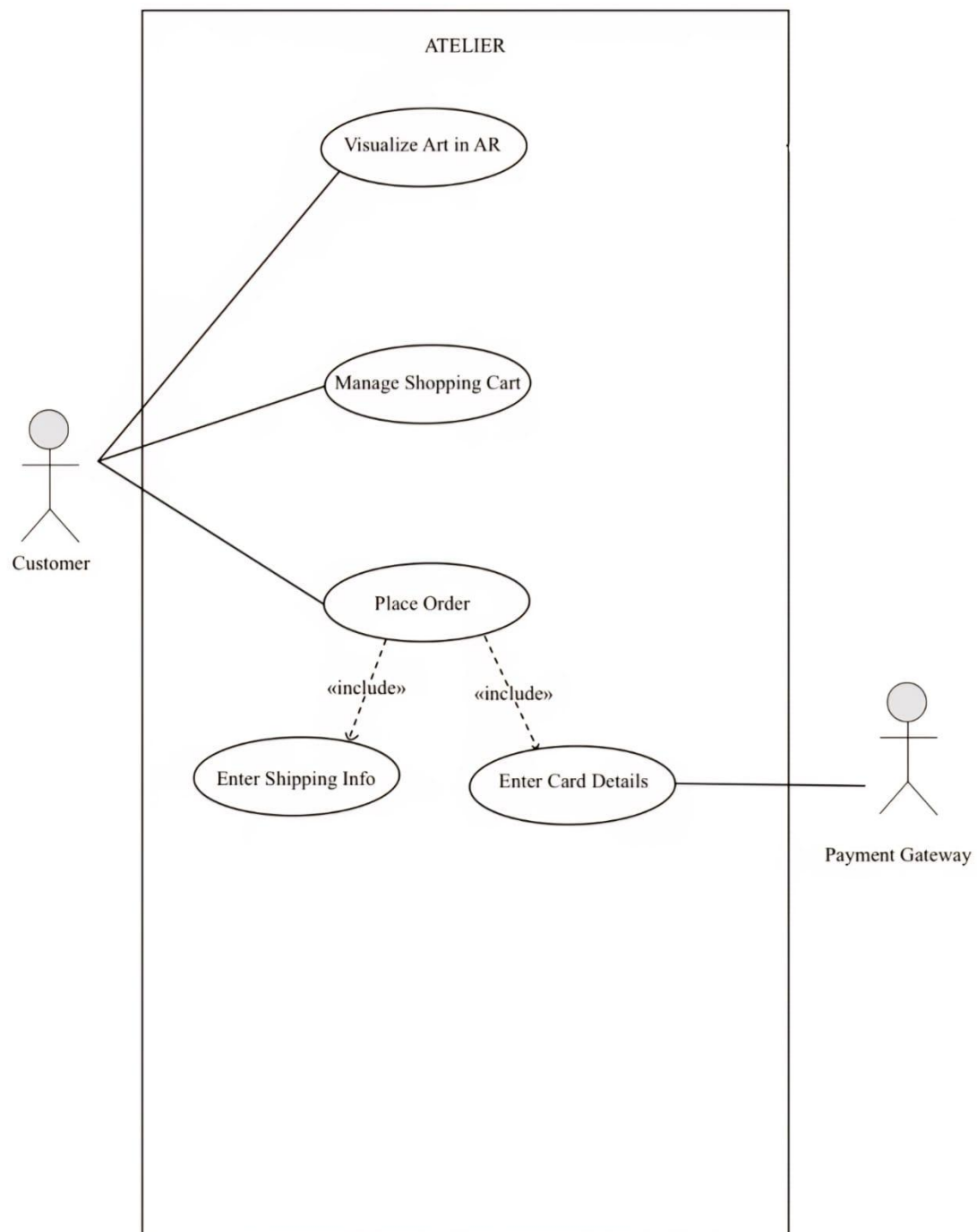
1.3 Non-Functional Requirements:

NFR-2: The website should be useable on both mobile phones and desktop computers

NFR-4: The system shall maintain full functionality on Chrome, Firefox, Safari, and Edge.

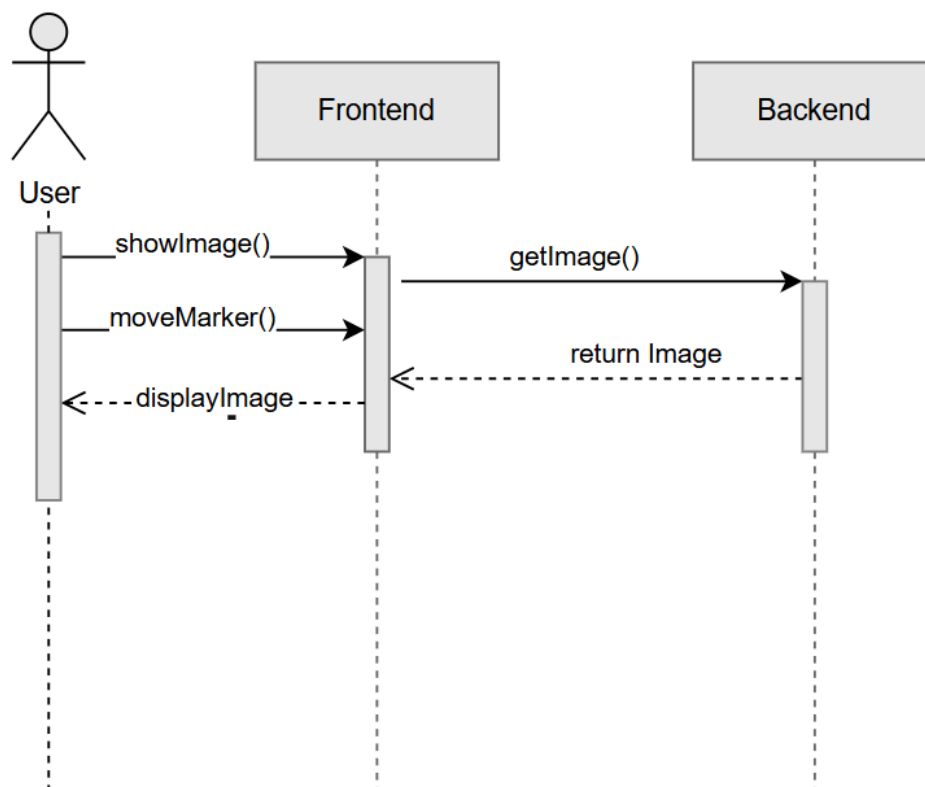
2 Diagrams

2.1 Use case:

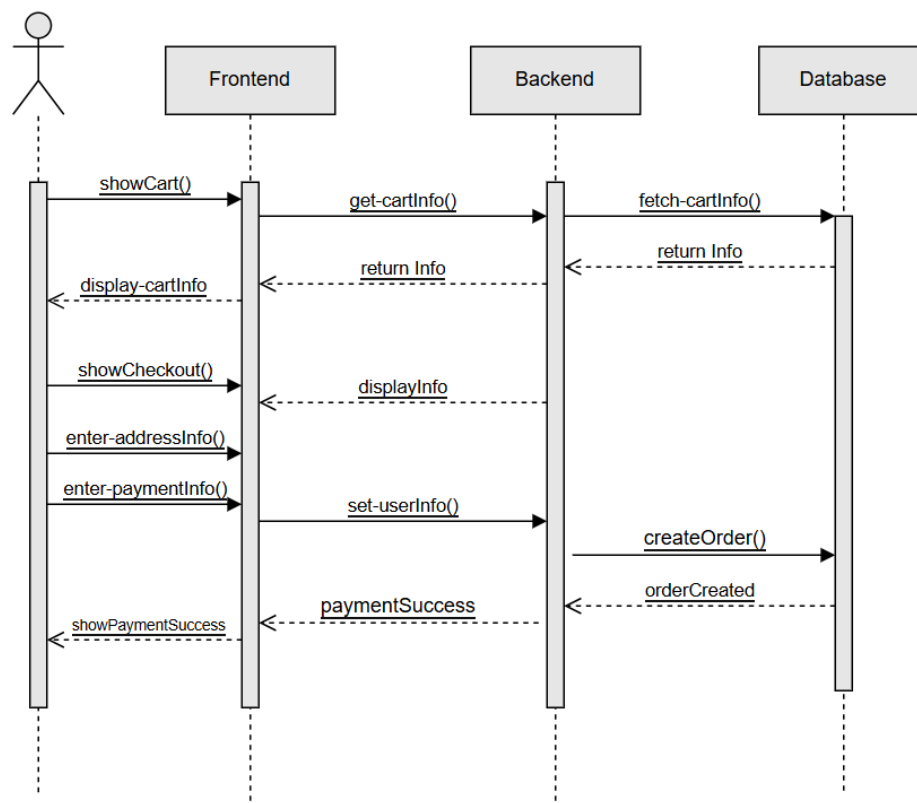


2.2 Sequence Diagram:

2.1 AR image display:



2.2 place order:



3 Stories:

3.1 Visualize Art in AR

Component Name: Web-AR

Story Name: Visualize Art in AR

Story Sequence No: 01

Story Short Description: Users can visualize paintings in their environment using AR.

Story Long Description: The user clicks the AR icon on a painting product page to open a new tab that activates the device camera, allowing them to visualize the painting in their environment using marker-based AR.

3.2 Manage Shopping Cart

Component Name: Cart

Story Name: Manage Shopping Cart

Story Sequence No: 02

Story Short Description: Users can add, view, and remove products from their shopping cart.

Story Long Description: The user adds products by clicking the cart icon, views all items grouped by artisan in the cart page with calculated totals, and removes products by clicking the cart icon again.

3.3 Place Order

Component Name: Checkout

Story Name: Place Order

Story Sequence No: 02

Story Short Description: Users can place an order for items in their cart.

Story Long Description: The user navigates checkout from the cart page, enters shipping information and card details, and completes the order after validation.

4 Meetings:

Sprint Meeting(s)

Project Name: Atelier

Project Members:

Remas Alsulami - Marwa Hadaddi - Reman Alamoudi - Ghaida Almeahmadi - Shumokh Alsharif

Sprint #2
Progress Check In Meeting
[18 November 2025]

Sprint Duration: ...

Scrum Master: Shumokh Alsharif

Client: Malak Alharbi

Pair Programmers:

Remas Alsulami - Marwa Hadaddi - Reman Alamoudi - Ghaida Almeahmadi - Shumokh Alsharif

Stories:

Components Name:

- Cart
- Checkout
- Web-AR

Follow-up meeting questions:

13. What has been completed since the last meeting?
14. What are you going to be working on next?
15. Do you have any issues/impediments?

Scrum's Master comments based on the above questions:

- 13- Tasks are distributed among the team and we started researching web AR
- 14- Complete the front end and back end for the cart and payment pages
- 15- Finding AR methods that are completable with different devices

Sprint Meeting(s)

Project Name: Atelier

Project Members:

Remas Alsulami - Marwa Hadaddi - Reman Alamoudi - Ghaida Almeahmadi - Shumokh Alsharif

Sprint #2

Progress Check In Meeting
[3 December 2025]

Sprint Duration: ...

Scrum Master: Shumokh Alsharif

Client: Malak Alharbi

Pair Programmers:

Remas Alsulami - Marwa Hadaddi - Reman Alamoudi - Ghaida Almeahmadi - Shumokh Alsharif

Stories:

Components Name:

- Cart
- Checkout
- Web-AR

Follow-up meeting questions:

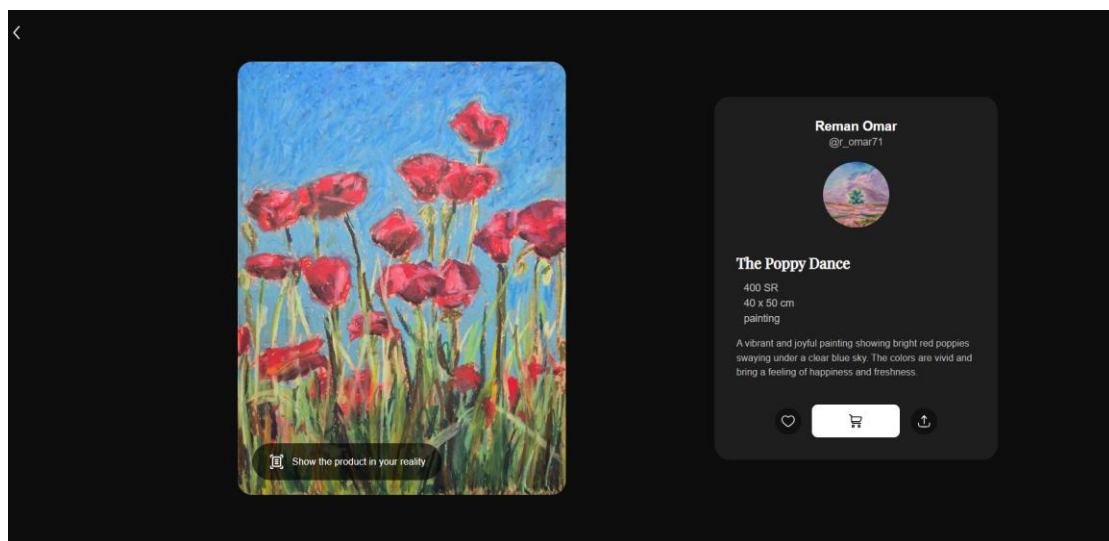
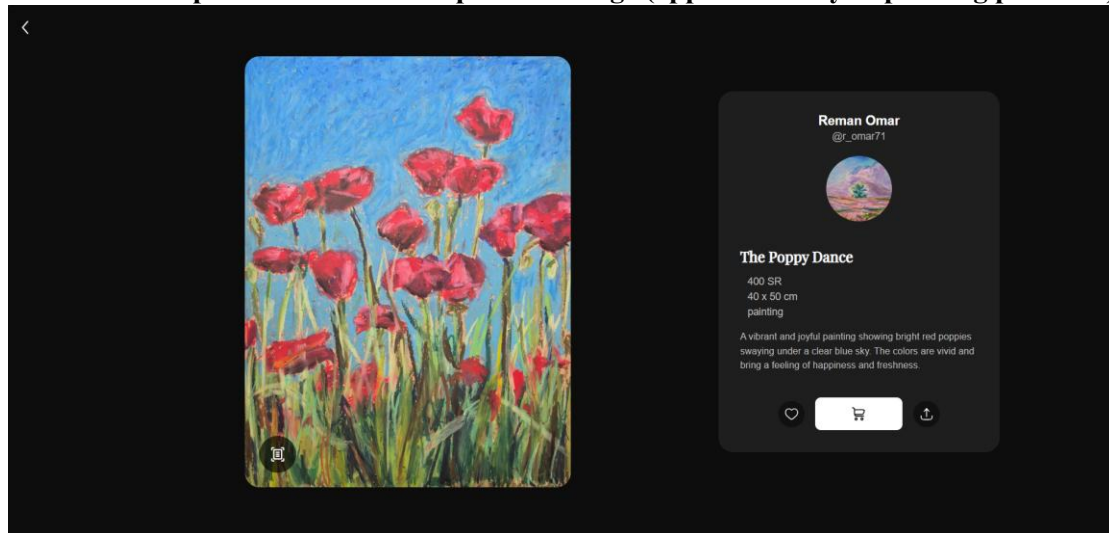
16. What has been completed since the last meeting?
17. What are you going to be working on next?
18. Do you have any issues/impediments?

Scrum's Master comments based on the above questions:

- 16- All new endpoints are added to the server, the frontend pages are complete, the web AR is fully working
- 17- Refining and integrating what has been built so far to work together seamlessly
- 18- None

5 Implementation in Web browser (UI):

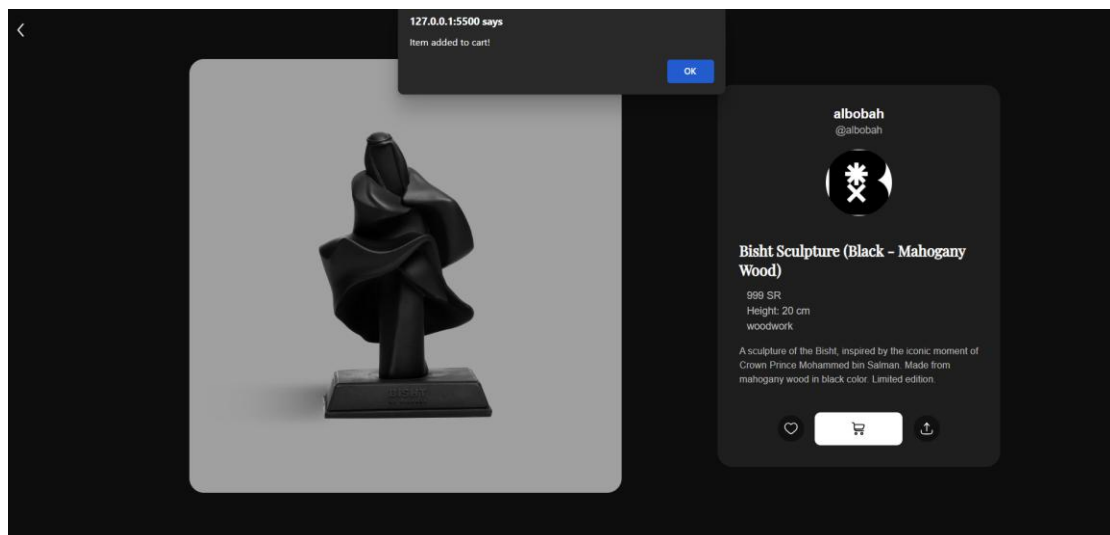
5.1 AR button positioned above the product image (applicable only to painting products)



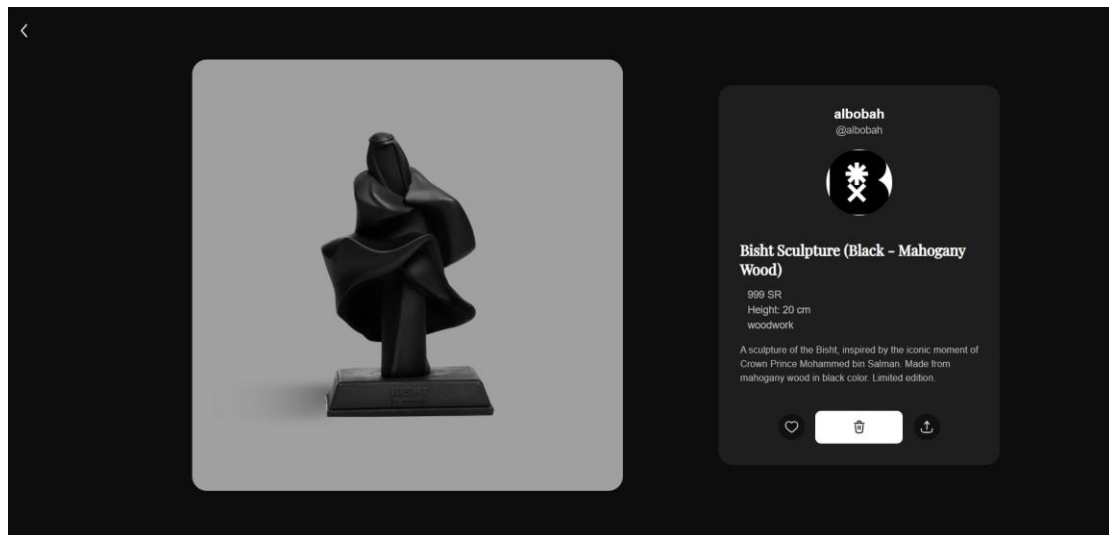
5.2 AR Feature



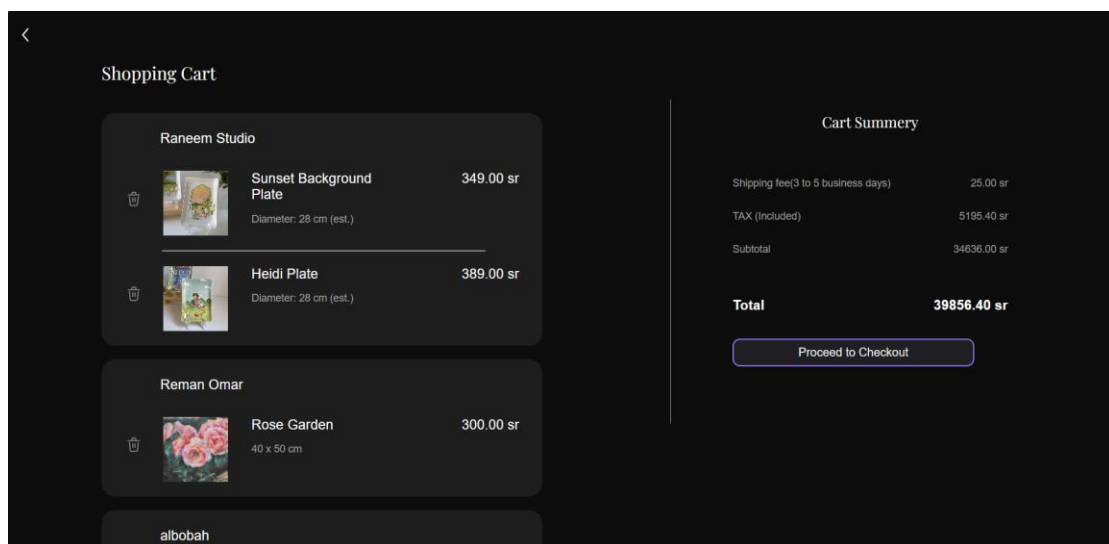
5.3 Add to cart button on product page



5.4 Delete from the cart button on product page



5.5 Cart page



5.6 Checkout page

[<](#) Checkout

Shipping Address

First Name

Enter first name

Last Name

Enter last name

Phone Number

055 000 0000

Address

City, District, Street...

Payment Method

Card Number

0000 0000 0000 0000

Expiry Date

MM/YY

CVV

123

Total Amount

41925.25 SR

Shipping Address

First Name

Enter first name

Last Name

Enter last name

Phone Number

055 000 0000

Address

City, District, Street...

Payment Method

Card Number

0000 0000 0000 0000

Expiry Date

MM/YY

CVV

123

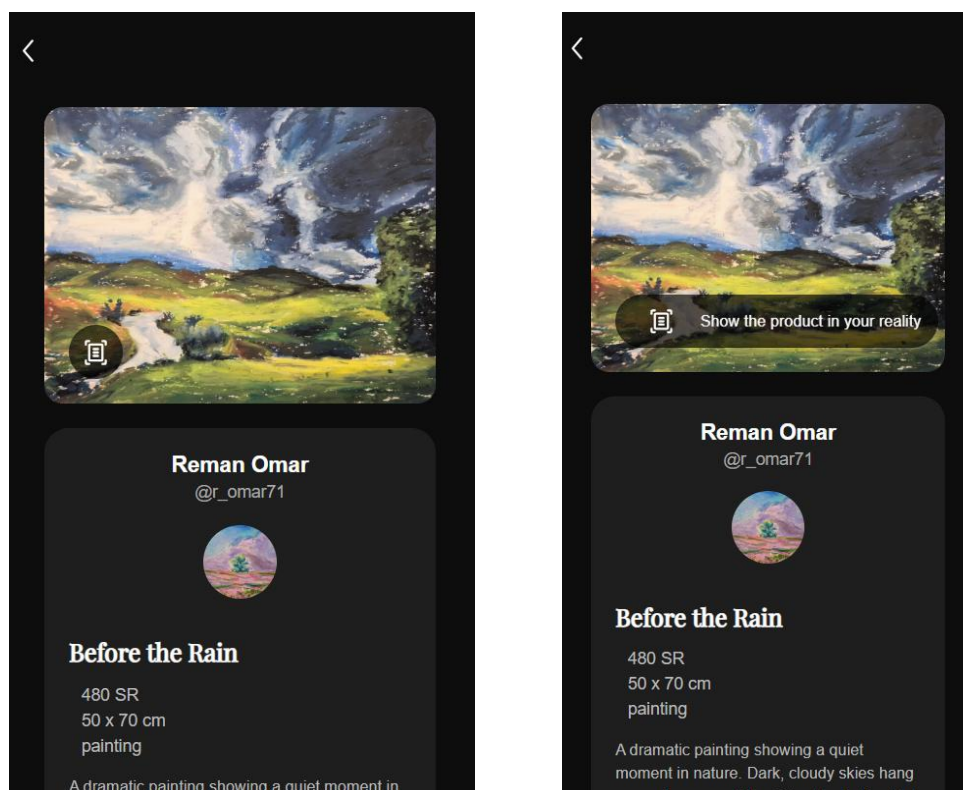
Total Amount

41925.25 SR

PAY NOW

6 Implementation in Mobile (UI):

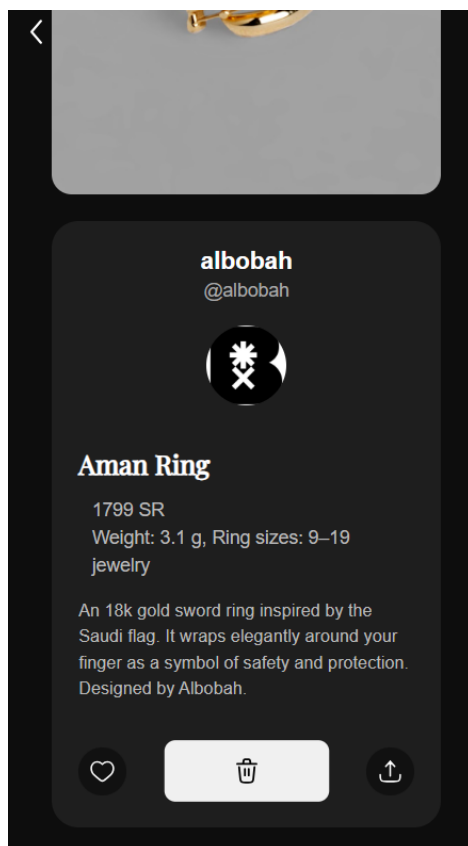
6.1 AR button positioned above the product image (applicable only to painting products)



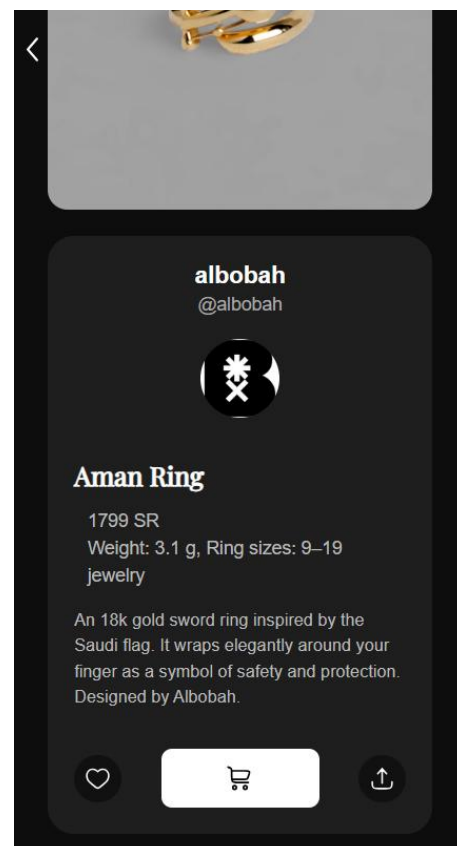
6.2 AR Feature



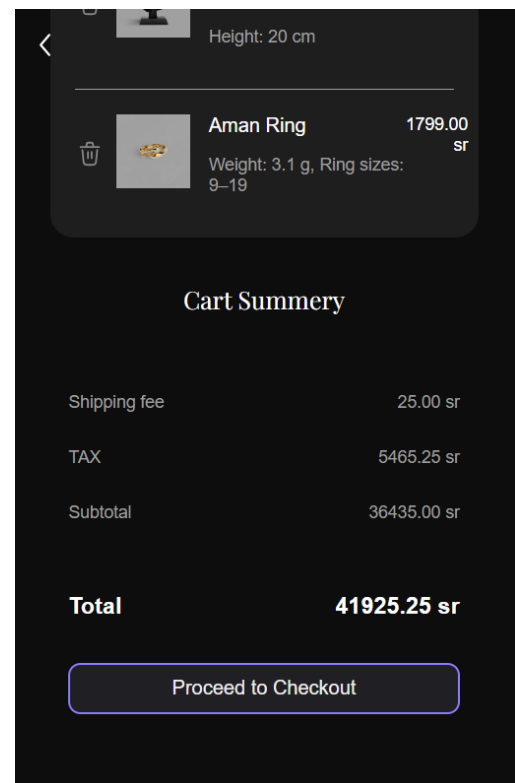
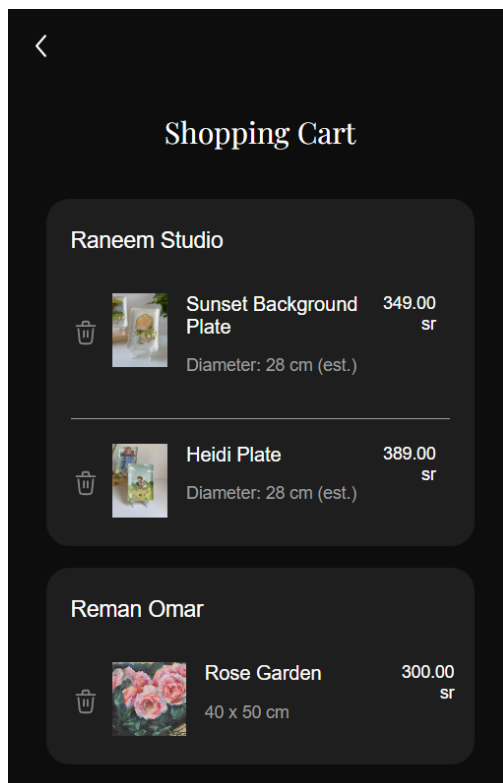
6.4 Delete from the cart button on product page



6.3 Add to cart button on product page



6.5 Cart page



6.6 Checkout page

< Checkout

Shipping Address

First Name
Enter first name

Last Name
Enter last name

Phone Number
055 000 0000

Address
City, District, Street...

Payment Method

Card Number
0000 0000 0000 0000

Expiry Date
MM/YY

CVV
123

Total Amount 41925.25 SR

PAY NOW

7 Test Cases

Test Case Name: Sprint 3- Cart

Test Case Id: Atelier - Cart

Test Case No.	Test Case Description	Expected Results	Outcome (Pass/Fail) Other (Comments))
TC001	Unregistered users try to add an item to the Cart	Product/collection won't be added and user will be prompted to log in first	Pass
TC002	Registered user clicks on the cart icon in product page	The product will be added to their cart	Pass
TC003	User clicks on the cart icon in home page	Transports user to Cart page	Pass
TC004	Registered user opens the cart page	All the products on their cart will be displayed	Pass
TC005	Unregistered user opens the cart page	Nothing is displayed and user's cart will be empty	Pass
TC006	Registered user clicks on the cart icon for an item already in the cart	Product will be removed from the cart	Pass
TC007	User change window size	The layout adjusts accordingly	Pass
TC008	User clicks on any product image on the cart's page.	User is navigated to the product's specific page (Product page).	Pass
TC009	User clicks the "Back" button from the cart page.	User successfully returns to the home page.	Pass
TC010	User clicks the "Back" button from the product page.	User successfully returns to the cart Page.	Pass
TC011	User clicks on the checkout button	User goes to the checkout page	Pass

Test Case Name: Sprint 3- Checkout

Test Case Id: Atelier – Checkout

Test Case No.	Test Case Description	Expected Results	Outcome (Pass/Fail) Other (Comments))
TC001	User forgets a required field	The required field will vibrate, and the user won't continue until it is filled	Pass
TC002	User places their order	A confirmation message is displayed on screen and cart is emptied	Pass
TC003	User change window size	The layout adjusts accordingly	Pass
TC004	User clicks the “Back” button from the checkout page.	User successfully returns to the cart page.	Pass

Test Case Name: Sprint 3- Web-AR

Test Case Id: Atelier – Web-AR

Test Case No.	Test Case Description	Expected Results	Outcome (Pass/Fail) Other (Comments))
TC001	User opens product a page (painting)	Web-AR icon is found over the image on the bottom left	Pass
TC002	User hovers over the Web-Air icon	Icon expands over hovering	Pass
TC003	User clicks on Web-Air icon	A new tab opens for Web-AR	Pass
TC004	The camera detects the marker	The chosen painting is displayed with AR	Pass
TC005	User opens a product page (not paining)	The AR button is disabled	Pass
TC006	No marker found	The Web-AR will not display anything	Pass

Code:

1 Backend:

1.1 server:

```
server.js > app.post("/commission") callback > attachments

1
2   const mongoose = require("mongoose");
3   const express = require("express");
4   const bcrypt = require('bcrypt');
5   const cors = require('cors');
6   const jwt = require('jsonwebtoken');
7
8   const product = require("./models/product");
9   const artist = require("./models/artist");
10  const User = require('./models/user');
11  const Commission = require('./models/commission');
12
13  const JWT_SECRET = 'AA.201424';
14  const auth = require('./middleware/auth');
15
16
17  mongoose
18    .connect("mongodb+srv://ghaida:GS.201424@cluster.cakgpc.mongodb.net/?retryWrites=true&w=majority&appName=Cluster")
19    .then(() => {
20      console.log(" connected successfully");
21    })
22    .catch((error) => {
23      console.log(" error with connecting to the DB", error);
24    });
25
26
27
28  const app = express()
29
30  app.use(express.json())
31  app.use(cors())
32
33  app.get("/", (req,res)=>{
34    res.send("Welcom to the Atelier server root!!!")
35  })
36
37
38  app.get("/products", async (req, res) =>{
39
40    try{
41      const products = await product.find()
42      console.log("Fething products completed!" )
43      res.json(products)
44      return
45    }catch (error) {
46      console.log("error while fetching products ", error)
47      return res.send("error ")
48    }
49
50  })
51
52
53
54  app.get("/products/category/:category", async (req, res) =>{
55
56    try{
57      const catagory = req.params.category
58
59      const productsByCategory = await product.find({ category: catagory }).lean();
60
61      if(!productsByCategory){
62        return res.send("No product was found for this category")
63      }
64    }
```



```
64
65 // const collections = await product.distinct("collections", { category })
66 // const collections = [...new Set(productsByCategory.map(p => p.collections || p.collections?.[0]))]
67 const collections = [
68   ...new Set(
69     productsByCategory.map(p => Array.isArray(p.collections) ? p.collections[0] : p.collections)
70   )
71 ];
72
73
74 // res.json(productsByCategory)
75
76 res.json({
77   collections: collections,
78   products: productsByCategory
79 });
80 console.log("Fething products and collections by category is completed!")
81 return
82
83 }catch (error) {
84   console.log("error while fetching products by category", error)
85   return res.send("error")
86 }
87
88
89 })
90
91
```

```
92 app.get("/products/id/:productId", async (req, res) =>{
93   const id = req.params.productId
94
95   try{
96
97     const productById = await product.findById(id)
98     if(!productById){
99       return res.send("No product was found with this id")
100     }
101
102     // res.json(productById)
103
104     const artistById = await artist.findOne({ artistId: productById.artistId })
105     if(!artistById){
106       return res.send("No artist was found with this id")
107     }
108
109     res.json({
110       product: productById,
111       artist: artistById
112     })
113     console.log("Fething products by ID is completed!")
114     return
115
116   }catch (error) {
117     console.log("error while fetching products by ID")
118     return res.send("error")
119   }
120
121
122 })
123
```

```
124
125 app.get("/products/collection/:collection", async (req, res) =>{
126
127   try{
128     const collection = req.params.collection
129
130     const collectionProducts= await product.find({ collections: collection})
131     if(!collectionProducts){
132       | return res.send("No product was found in this collection")
133     }
134
135     // res.json(collectionProducts)
136     const artistById = await artist.findOne({ artistId: collectionProducts[0].artistId });
137     if (!artistById) {
138       | return res.send("No artist was found for this collection");
139     }
140
141     res.json({
142       products: collectionProducts,
143       artist: artistById
144     });
145
146     console.log("Fething products in the collection is completed!" )
147     return
148   }catch (error) {
149     console.log("error while fetching the collection")
150     return res.send("error")
151   }
152 }
153
154 })
155
156
```

```
157 app.get("/collections", async (req, res) => {
158   try {
159     const collections = await product.distinct("collections");
160     if (!collections || collections.length === 0) {
161       | return res.send("No collections found");
162     }
163
164     res.json(collections);
165     console.log("Fetching all unique collections completed!");
166   } catch (error) {
167     console.log("Error while fetching collections:", error);
168     res.send("error");
169   }
170 });
171
172
173 app.get("/collections/artist/:artistId", async (req, res) => {
174   try {
175
176     const artistId = req.params.artistId;
177     const collections = await product.distinct("collections", { artistId: artistId });
178
179
180     if (!collections || collections.length === 0) {
181       | return res.send("No collections found for this artist");
182     }
183
184     //fetch product images
185     const results = [];
186
187     for (const collection of collections) {
188       | const productsInCollection = await product
189       | find({ artistId: collection.collections })
190     }
191   }
192 }
```

```
188     for (const collection of collections) {
189         const productsInCollection = await product
190             .find({ artistId, collections: collection })
191             .limit(2)
192             .select("images");
193
194
195         const images = productsInCollection.map(p => p.images[0]).filter(Boolean);
196
197         results.push({
198             collection,
199             images
200         });
201     }
202
203     res.json(results);
204     console.log("Fetching collections by artist completed!");
205 } catch (error) {
206     console.log("Error while fetching collections by artist: ", error);
207     res.send("error");
208 }
209
210 });
211
212
213
214 app.get("/products/artistId/:artistId", async (req, res) =>{
215
216     try{
217         const artistId = req.params.artistId
218
219         const productsByArtisan = await product.find({ artistId: artistId })
```

```
214 app.get("/products/artistId/:artistId", async (req, res) =>{
215
216     try{
217         const artistId = req.params.artistId
218
219         const productsByArtisan = await product.find({ artistId: artistId })
220
221         if (!productsByArtisan || productsByArtisan.length === 0) {
222             return res.send("No product was found for this artist")
223         }
224
225
226         const artistInfo = await artist.findOne({ artistId })
227
228         if (!artistInfo) {
229             return res.send("No artist info found for this artist")
230         }
231
232         res.json({
233             artist: artistInfo,
234             products: productsByArtisan
235         })
236
237
238
239     } catch (error) {
240         console.log("error while fetching products by artisan", error)
241         return res.send("error")
242     }
243
244
245 })
246
```

```
247
248 app.get("/products/artistId/available/:artistId", async (req, res) => {
249   try {
250     const artistId = req.params.artistId;
251
252     const inStockProducts = await product.find({
253       artistId: artistId,
254       availability: "In Stock"
255     });
256
257     res.json(inStockProducts);
258   } catch (error) {
259     console.log("Error while fetching in-stock products by artist:", error);
260     return res.send("Error fetching in-stock products");
261   }
262 });
263
264
265
266 // Wishlist Endpoints
267 const { Types: { ObjectId } } = mongoose;
268
269 function isValidObjectId(id) {
270   return ObjectId.isValid(id);
271 }
272
273 /**
274  * retrieves the full wishlist
275  */
276 app.get("/users/:userId/wishlist", async (req, res) => {
277   try {
278     const { userId } = req.params;
279     if (!isValidObjectId(userId)) {
```

```
278     const { userId } = req.params;
279     if (!isValidObjectId(userId)) {
280       return res.status(400).json({ message: "Invalid userId" });
281     }
282
283     const user = await User.findById(userId, { wishlist: 1, _id: 0 }).lean();
284     if (!user) return res.status(404).json({ message: "User not found" });
285
286     return res.status(200).json(Array.isArray(user.wishlist) ? user.wishlist : []);
287   } catch (err) {
288     console.error("GET /wishlist error:", err);
289     return res.status(500).json({ message: "Error fetching wishlist" });
290   }
291 });
292
293 /**
294  * retrieves only the productIds from the wishlist
295  */
296 app.get("/users/:userId/wishlist/products", async (req, res) => {
297   try {
298     const { userId } = req.params;
299     if (!isValidObjectId(userId)) {
300       return res.status(400).json({ message: "Invalid userId" });
301     }
302
303     const user = await User.findById(userId, { wishlist: 1, _id: 0 }).lean();
304     if (!user) return res.status(404).json({ message: "User not found" });
305
306     const productIds = (user.wishlist || []).
307       .filter(item => item && item.product)
308       .map(item => item.product);
309
310     return res.status(200).json(productIds);
311   } catch (err) {
312     console.error("GET /wishlist/products error:", err);
313     return res.status(500).json({ message: "Error fetching wishlist products" });
314   }
315 });
```

```
310     return res.status(200).json(productIds);
311   } catch (err) {
312     console.error("GET /wishlist/products error:", err);
313     return res.status(500).json({ message: "Error fetching product wishlist" });
314   }
315 });
316
317 /**
318  * adds/removes a product from the wishlist
319  */
320 app.put("/users/:userId/wishlist/products/toggle", async (req, res) => {
321   try {
322     const { userId } = req.params;
323     const { productId, collection } = req.body;
324
325     if (!isValidObjectId(userId) || !isValidObjectId(productId)) {
326       return res.status(400).json({ message: "Invalid userId or productId" });
327     }
328
329     // to verify product exists
330     const exists = await product.exists({ _id: productId });
331     if (!exists) return res.status(404).json({ message: "Product not found" });
332
333     const user = await User.findById(userId);
334     if (!user) return res.status(404).json({ message: "User not found" });
335
336     const list = user.wishlist || [];
337     const index = list.findIndex(
338       item => item.product && item.product.toString() === productId
339     );
340
341     let toggled;
```

```
342
343     if (index > -1) {
344       // remove
345       list.splice(index, 1);
346       toggled = "removed";
347     } else {
348       // add
349       list.push({
350         product: productId,
351         collection: collection || null,
352         dateAdded: new Date()
353       });
354       toggled = "added";
355     }
356
357     user.wishlist = list;
358     await user.save();
359
360     return res.status(200).json({
361       toggled,
362       wishlist: user.wishlist
363     });
364   } catch (err) {
365     console.error("PUT /wishlist/products/toggle error:", err);
366     return res.status(500).json({ message: "Error toggling product wishlist" });
367   }
368 });
369
370 /**
371  * retrieves wishlist collections with counts
372  */
```

```
402 /**  
403  * adds/removes a collection bookmark from the wishlist  
404  */  
405 app.put("/users/:userId/wishlist/collections/toggle", async (req, res) => {  
406   try {  
407     const { userId } = req.params;  
408     const { collection } = req.body;  
409  
410     if (!isValidObjectId(userId)) {  
411       return res.status(400).json({ message: "Invalid userId" });  
412     }  
413  
414     if (!collection || typeof collection !== "string") {  
415       return res.status(400).json({ message: "Collection is required" });  
416     }  
417  
418     const user = await User.findById(userId);  
419     if (!user) return res.status(404).json({ message: "User not found" });  
420  
421     const list = user.wishlist || [];  
422  
423     // look for bookmark (product: null)  
424     const index = list.findIndex(  
425       (item) =>  
426         item &&  
427         (item.product === null || item.product === undefined) &&  
428         item.collection === collection  
429     );  
430  
431     let toggled;  
432
```

```
433     if (index > -1) {  
434       //if found + remove bookmark  
435       list.splice(index, 1);  
436       toggled = "removed";  
437     } else {  
438       //if not found + add bookmark  
439       list.push({  
440         product: null,  
441         collection,  
442         dateAdded: new Date(),  
443       });  
444       toggled = "added";  
445     }  
446  
447     user.wishlist = list;  
448     await user.save();  
449  
450     return res.status(200).json({  
451       toggled,  
452       wishlist: user.wishlist,  
453     });  
454   } catch (err) {  
455     console.error("PUT /wishlist/collections/toggle error:", err);  
456     return res  
457       .status(500)  
458       .json({ message: "Error toggling collection wishlist" });  
459   }  
460 });  
461
```

```
462 // End Wishlist Endpoints
463
464 app.post("/product", async (req, res) => {
465   try {
466     const newProduct = new product(req.body);
467
468     await newProduct.save();
469
470     res.json(newProduct);
471   } catch (error) {
472     res.status(400).json({ message: error.message });
473   }
474 });
475
476
477 // Create new artist
478 app.post("/artists", async (req, res) => {
479   try {
480     const newArtist = new artist(req.body);
481     await newArtist.save();
482     res.json(newArtist);
483   } catch (error) {
484     res.status(400).json({ message: error.message });
485   }
486 });
487
488
489 const { sendEmail } = require('./email');
490 const user = require("../models/user");
491
492 app.post("/commission", async (req, res) => {
493   try {
494
```

```
495     const {
496       type,
497       dimensions,
498       attachment,
499       description,
500       phoneNumber,
501       country,
502       city,
503       isGift,
504       artistname,
505       artistEmail,
506       userEmail,
507       username
508     } = req.body;
509
510
511     const newCommission = {
512       type,
513       dimensions,
514       attachment,
515       description,
516       phoneNumber,
517       country,
518       city,
519       isGift,
520       artistname,
521       artistEmail,
522       userEmail,
523       dateSubmitted: new Date()
524     }
525
```

```

559 // user email
560 try {
561     await sendEmail({
562         to: username,
563         subject: "Your Commission Has Been Sent",
564         template: 'userView',
565         context: {
566             username: username,
567             artistname: artistname
568         },
569         attachments: [
570             {
571                 filename: 'Email_icon.svg',
572                 path: __dirname + '/views/Group6.svg',
573                 cid: 'mail@atelier'
574             }
575         ]
576     }) catch (error) {
577         console.error("Email error:", error);
578     }
579
580
581
582 /*
583 try {
584     await sendEmail({
585         to: userEmail,
586         subject: "Your Commission Has Been Sent",
587         template: 'userView',
588         context: { username },
589         attachments: [

```



```
606     res.status(201).json({
607       | message: "Commission request received successfully! ",
608       | requestData: newCommission
609     })
610
611     // afte the comission schema is done
612     // const savedRequest = await CommissionRequest.create(newCommission);
613     // await sendCommissionEmail(newCommission.artistEmail, newCommission);
614     // res.status(201).json({
615     //   | message: "Commission request received successfully!",
616     //   | requestData: savedRequest
617     // });
618
619
620   } catch (error) {
621     console.error("Error while submitting commission request: ", error)
622     res.status(500).json({ message: "Error while submitting commission" })
623   }
624 }
625
626
627 app.post('/api/register', async (req, res) => {
628   const { username, email, password } = req.body;
629
630   try {
631     const existingUser = await User.findOne({ $or: [{ email }, { username }] });
632
633     if (existingUser) {
634       const field = existingUser.email === email ? 'Email' : 'Username';
635       return res.status(400).json({ message: `${field} is already in use.` });
636     }
637
638     const newUser = new User({ username, email, password });
639     await newUser.save();
640
641     res.status(201).json({
642       | message: 'Registration successfull',
643       | user: {
644       |   | id: newUser._id,
645       |   | username: newUser.username,
646       |   | email: newUser.email
647       | }
648     });
649
650   } catch (error) {
651     console.error('Registration Error:', error.message);
652     res.status(500).json({ message: 'Server error during registration.' });
653   }
654 });
655
656 app.post('/api/login', async (req, res) => {
657   const { email, password } = req.body;
658
659   try {
660     const user = await User.findOne({ $or: [{ email }, { username: email }] });
661
662     if (!user) {
663       return res.status(401).json({ message: 'Authentication failed. Invalid username or password.' });
664     }
665
666     const isMatch = await bcrypt.compare(password, user.password);
667
668     if (!isMatch) {
669       return res.status(401).json({ message: 'Authentication failed. Invalid username or password.' });
670     }
671
672     const token = jwt.sign({ id: user._id }, process.env.JWT_SECRET, { expiresIn: '1h' });
673     res.status(200).json({ message: 'Login successful', token });
674   } catch (error) {
675     console.error('Login Error:', error.message);
676     res.status(500).json({ message: 'Server error during login.' });
677   }
678 });
```

```
637
638   const newUser = new User({ username, email, password });
639   await newUser.save();
640
641   res.status(201).json({
642     | message: 'Registration successfull',
643     | user: {
644     |   | id: newUser._id,
645     |   | username: newUser.username,
646     |   | email: newUser.email
647     | }
648   });
649
650 } catch (error) {
651   console.error('Registration Error:', error.message);
652   res.status(500).json({ message: 'Server error during registration.' });
653 }
654 });
655
656 app.post('/api/login', async (req, res) => {
657   const { email, password } = req.body;
658
659   try {
660     const user = await User.findOne({ $or: [{ email }, { username: email }] });
661
662     if (!user) {
663       return res.status(401).json({ message: 'Authentication failed. Invalid username or password.' });
664     }
665
666     const isMatch = await bcrypt.compare(password, user.password);
667
668     if (!isMatch) {
669       return res.status(401).json({ message: 'Authentication failed. Invalid username or password.' });
670     }
671
672     const token = jwt.sign({ id: user._id }, process.env.JWT_SECRET, { expiresIn: '1h' });
673     res.status(200).json({ message: 'Login successful', token });
674   } catch (error) {
675     console.error('Login Error:', error.message);
676     res.status(500).json({ message: 'Server error during login.' });
677   }
678 });
```

```
671
672   const token = jwt.sign(
673     { userId: user._id, email: user.email },
674     JWT_SECRET,
675     { expiresIn: '1d' }
676   );
677
678   res.status(200).json({
679     message: 'Login successful',
680     token,
681     user: {
682       id: user._id,
683       username: user.username,
684       email: user.email
685     }
686   });
687
688   } catch (error) {
689     console.error(error);
690     res.status(500).json({ message: 'Server error during login.' });
691   }
692
693   });
694
695
696
697   app.get("/users", async (req, res) =>{
698
699     try{
700       const users = await user.find()
701       console.log("Fetching users completed!")
702       res.json(users)
703       return
```

```
704     }catch (error) {
705       console.log("error while fetching users ", error)
706       return res.send("error ")
707     }
708
709   })
710
711
712   app.get("/user/:userId", async (req,res) =>{
713     const userId = req.params.userId
714     try{
715       const userinfo = await user.findById(userId).select("username email")
716
717       if (!userinfo){
718         return res.status(404).send("User not found");
719       }
720
721       res.send(userinfo)
722       console.log("User info fetched successfully")
723
724     }catch(error){
725       console.log("error while fetching a user info ", error)
726       res.status(500).send("Error while fetching user info")
727     }
728   })
729
730   app.post("/cart/add", async (req, res) => {
731     const { userId, productId, quantity } = req.body;
732
733     if (!userId || !productId) {
734       return res.status(400).json({ message: "Missing required fields: userId and productId are required" });
735     }
736   })
```

```
735 }
736
737 try {
738   const user = await User.findById(userId);
739
740   if (!user) {
741     return res.status(404).json({ message: "User not found" });
742   }
743
744   const qty = quantity && quantity > 0 ? quantity : 1;
745
746   const itemIndex = user.cart.findIndex(item => item.product.toString() === productId);
747
748   if (itemIndex > -1) {
749     user.cart[itemIndex].quantity += qty;
750   } else {
751     user.cart.push({
752       product: productId,
753       quantity: qty,
754       dateAdded: new Date()
755     });
756   }
757
758   await user.save();
759
760   res.status(200).json({
761     message: "Cart updated successfully",
762     cart: user.cart
763   });
764 } catch (error) {
765   console.error("Error in /cart/add:", error);
766   res.status(500).json({ message: "Error adding to cart" });
767 }
```

```
767   res.status(500).json({ message: "Error adding to cart" });
768 }
769 });
770
771 app.delete("/cart/remove", async (req, res) => {
772   const { userId, productId } = req.body;
773
774   if (!userId || !productId) {
775     return res.status(400).json({ message: "Missing userId or productId" });
776   }
777
778   try {
779     const user = await User.findById(userId);
780
781     if (!user) {
782       return res.status(404).json({ message: "User not found" });
783     }
784
785     const originalLength = user.cart.length;
786
787     user.cart = user.cart.filter(item => item.product.toString() !== productId);
788
789     if (user.cart.length === originalLength) {
790       return res.status(404).json({ message: "Product not found in cart" });
791     }
792
793     await user.save();
794
795     res.status(200).json({
796       message: "Item removed successfully",
797       cart: user.cart
798     });
799   } catch (error) {
800     console.error("Error in /cart/remove:", error);
801     res.status(500).json({ message: "Error removing item" });
802   }
803 }
```

```
799 } catch (error) {
800   console.error("Error in /cart/remove:", error);
801   res.status(500).json({ message: "Error removing item from cart" });
802 }
803 }
804 });
805
806 app.get('/cart/:userId', async (req, res) => {
807   try {
808     const { userId } = req.params;
809
810     const user = await User.findById(userId).populate({
811       path: 'cart.product',
812       model: 'product',
813       select: 'name images price dimensions artistId'
814     });
815
816     if (!user) {
817       return res.status(404).json({ message: 'User not found' });
818     }
819
820     let subTotal = 0;
821     const groupsObj = {};
822
823     for (const cartItem of user.cart) {
824       if (!cartItem.product) continue;
825
826       const product = cartItem.product;
827
828       const lineTotal = product.price;
829       subTotal += lineTotal;
830     }
```

```
831     const itemData = {
832       productId: product._id,
833       name: product.name,
834       picture: product.images?.[0] || '',
835       price: product.price,
836       dimensions: product.dimensions,
837       lineTotal
838     };
839
840     // Use findOne with custom artistId
841     const artistDoc = await artist.findOne({ artistId: String(product.artistId) });
842     const artistName = artistDoc?.name || "Unknown Artist";
843
844     if (!groupsObj[artistName]) {
845       groupsObj[artistName] = [];
846     }
847
848     groupsObj[artistName].push(itemData);
849   }
850 }
851
852 const groupedCart = Object.keys(groupsObj).map(artist => ({
853   artist: artist,
854   items: groupsObj[artist]
855 }));
856
857 const TAX_RATE = 0.15;
858 const taxAmount = subTotal * TAX_RATE;
859 const shipping = 25;
860 const total = shipping + subTotal + taxAmount;
861
862 res.status(200).json({
```

```
889     if (!user) {
890       | | return res.status(404).json({ message: "User not found" });
891     }
892
893     // Key Action: Clear the entire cart array
894     user.cart = [];
895
896     await user.save();
897
898     res.status(200).json({
899       | | message: "Cart cleared successfully.",
900       | | cart: user.cart
901     });
902
903   } catch (error) {
904     | console.error("Error in /cart/clear:", error);
905     | res.status(500).json({ message: "Error clearing cart" });
906   }
907 });
908
909
910 app.post("/checkout", async (req, res) => {
911   const { userId } = req.body;
912
913   if (!userId) {
914     | return res.status(400).json({ message: "UserId is required" });
915   }
916
917   try {
918     const user = await User.findById(userId);
919
920     if (!user) {
```

```
921     | return res.status(404).json({ message: "User not found" });
922   }
923
924   user.cart = [];
925
926   await user.save();
927
928   res.status(200).json({ message: "Checkout successful and cart cleared!" });
929   console.log(`Cart cleared for user: ${userId}`);
930
931 } catch (error) {
932   | console.error("Error during checkout:", error);
933   | res.status(500).json({ message: "Server error during checkout" });
934 }
935 });
936
937
938 app.listen(3000, () =>{
939   | console.log("I'm listening in the port 3000")
940 })
941
942
```

1.2 AR:

```
1 <!doctype html>
2 <html>
3 <head>
4
5 <script src="https://aframe.io/releases/1.2.0/aframe.min.js"></script>
6 <script src="https://cdn.jsdelivr.net/gh/AR-js-org/AR.js@3.3.2/aframe/build/aframe-ar.js"></script>
7 </head>
8
9 <body style="margin: 0; overflow: hidden;">
10 <a-scene
11   embedded
12   vr-mode-ui="enabled: false"
13   renderer="logarithmicDepthBuffer: true;"
14   arjs="trackingMethod: best; sourceType: webcam; debugUIEnabled: true;">
15
16
17   <a-marker preset="hiro">
18     | <!--<a-box position="0 0.5 0" color="red"></a-box> -->
19
20     <!--<a-marker type="pattern" url="assets/logo-marker.patt"> -->
21     <a-plane
22       id="ar-image"
23       src="ar-test.jpg"
24       height="1"
25       width="1.5"
26       position="0 0.5 0"
27       rotation="-90 0 0"
28     ></a-plane>
29   </a-marker>
30
31   <a-entity camera></a-entity>
32 </a-scene>
33
```

```
33
34
35 <script>
36
37   function getGoogleDriveDirectUrl(url) {
38     if (url.includes('drive.google.com')) {
39       const match = url.match(/(?:&id=([^\&]+))/);
40       if (match) {
41         const fileId = match[1];
42         return `https://lh3.googleusercontent.com/d/${fileId}`;
43       }
44     }
45     return url;
46   }
47
48   window.addEventListener('load', () => {
49     const urlParams = new URLSearchParams(window.location.search);
50     let imageUrl = urlParams.get('img');
51
52     if (imageUrl) {
53       imageUrl = getGoogleDriveDirectUrl(imageUrl);
54       console.log('Loading AR image:', imageUrl);
55
56       const arImage = document.getElementById('ar-image');
57       arImage.setAttribute('src', imageUrl);
58     }
59   });
60
61 </script>
62
63 </body>
```

2 Frontend:

```
index.html <!-- index.html -->
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4 <title>Home Page</title>
5 </head>
6 <body>
7 <div class="container">
8 <div class="row">
9 <div class="col-md-12">
10 <div class="text-center">
11 <img alt="Logo" data-bbox="150 300 250 350"/>
12 </div>
13 </div>
14 </div>
15 </div>
16 </body>
17 </html>

index.html <!-- index.html -->
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4 <title>Home Page</title>
5 </head>
6 <body>
7 <div class="container">
8 <div class="row">
9 <div class="col-md-12">
10 <div class="text-center">
11 <img alt="Logo" data-bbox="150 300 250 350"/>
12 </div>
13 </div>
14 </div>
15 </div>
16 </body>
17 </html>
```

```
index.html <!-- index.html -->
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4 <title>Checkout | Ateiler</title>
5 </head>
6 <body>
7 <div class="container">
8 <div class="row">
9 <div class="col-md-12">
10 <div class="text-center">
11 <img alt="Logo" data-bbox="150 300 250 350"/>
12 </div>
13 </div>
14 </div>
15 </div>
16 </body>
17 </html>

index.html <!-- index.html -->
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4 <title>Checkout | Ateiler</title>
5 </head>
6 <body>
7 <div class="container">
8 <div class="row">
9 <div class="col-md-12">
10 <div class="text-center">
11 <img alt="Logo" data-bbox="150 300 250 350"/>
12 </div>
13 </div>
14 </div>
15 </div>
16 </body>
17 </html>
```

```

product.html X
1 <DOCTYPE html>
2 <html lang="en">
3
4 <head>
5   <meta charset="UTF-8">
6   <meta name="viewport" content="width=device-width">
7   <title>Product Page</title>
8   <link href="https://fonts.googleapis.com/css2?family=Playfair+Display&display=swap" rel="stylesheet">
9   <link rel="stylesheet" href="css/product.css">
10 </head>
11
12 <body>
13   <!-- Header with back button -->
14   <header class="page_header">
15     <a href="#" class="back_icon" onclick="history.back()">
16       
17     </a>
18   </header>
19
20   <main class="product_page">
21     <!-- Product image section -->
22     <section class="product_image">
23       <img class="main_product_img" alt="Artwork">
24     </section>
25
26     <!-- AR Button -->
27     <button class="ar_button">
28       
29     </button>
30
31     <div class="ar_text">Show the product in your reality</div>
32   </main>
33
34   <!-- Product details card -->
35   <section class="product_card">
36
37     <!-- Artisan info -->
38     <div class="artisan_account">
39       <div>
40         <div class="username"></div>
41         <img id="artisan_avatar" alt="artisan_avatar">
42       </div>
43     </div>
44
45     <!-- Product information -->
46     <div class="product_details">
47       <div class="title"></div>
48       <div class="price"></div>
49     </div>
50
51   </section>
52
53   <!-- Footer -->
54   <div class="page_footer"></div>
55 </body>
56 </html>

```

```

product.js X
1 // ===== PRODUCT & ARTIST LOADING =====
2
3 // API endpoints
4 const API_PRODUCT_BY_ID = "https://stellar-badu-render.com/products/id/";
5 const API_COLLECTION_BY_SLUG = "https://stellar-badu-render.com/products/collection/";
6
7 const pick = (...xs) => xs.find(v => v !== undefined && v !== null && String(v).trim() !== "");
8
9 // Extract artist info
10 function extractArtist(o) {
11   const obj = o.artist || o.artistData || {};
12   return {
13     name: pick(o.artistName, obj.name),
14     username: pick(o.artistUsername, obj.username),
15     avatar: pick(o.artistAvatar, obj.profilePic, obj.avatarUrl, obj.avatar),
16   };
17 }
18
19 // Render the product page from a product object
20 function renderProduct(o) {
21   // Get our elements
22   const img = document.querySelector(".product_image img");
23   const title = document.querySelector(".title");
24   const price = document.querySelector(".price");
25   const size = document.querySelector(".size");
26   const medium = document.querySelector(".medium");
27   const desc = document.querySelector(".description");
28   const name = document.querySelector(".artisan_account .name");
29   const user = document.querySelector(".artisan_account .username");
30   const avatar = document.querySelector(".artisan_account img");
31   const productId = document.querySelector(".artisan_account .id");
32
33   // No product fields
34   const images = pick(
35     Array.isArray(o.images) && (o.images[0] || o.images[0]),
36     o.image, o.image1, o.photo, o.url
37   );
38   const title = pick(o.name, p.title, "Untitled");
39   const price = pick(o.price, p.cost);
40   const size = pick(o.size, p.dimensions, p.dimension);
41   const medium = pick(o.medium, p.material, p.category);
42   const desc = pick(o.description, p.details, "");
43
44   // Hide AR button visibility by category
45   const arButton = document.querySelector(".ar_button");
46   if (o.category) {
47     if (p.category === "painting") {
48       arButton.style.display = "flex";
49     } else {
50       arButton.style.display = "none";
51     }
52   }
53 }

```

```

profile.html X
1 <DOCTYPE html>
2 <html lang="en">
3
4 <head>
5   <meta charset="UTF-8">
6   <meta name="viewport" content="width=device-width">
7   <title>Artisan Profile</title>
8   <link href="https://fonts.googleapis.com/css2?family=Playfair+Display&display=swap" rel="stylesheet">
9   <link rel="stylesheet" href="css/profile.css">
10 </head>
11
12 <body>
13   <!-- Header with back button -->
14   <header class="page_header">
15     <a href="#" class="back_icon" onclick="history.back()">
16       
17     </a>
18   </header>
19
20   <!-- Artisan info -->
21   <div class="artisan_info">
22     <div class="name"></div>
23     <div class="username"></div>
24     <div class="profilePic"><img src="" alt=""></div>
25
26     <div class="link">
27       <a href="#" target="blank">
28         
29       </a>
30     </div>
31
32     <div class="description"></div>
33   </div>
34
35   <!-- to commission page -->
36   <button class="commission" onclick="gotoCommission()">
37     Commission
38   </button>
39
40   <!-- Footer -->
41   <div class="page_footer"></div>
42 </body>
43 </html>

```

```

profile.js X
1 document.addEventListener("DOMContentLoaded", () => {
2   const params = new URLSearchParams(window.location.search);
3   const productId = params.get("productId") || params.get("id");
4   if (!productId) return console.error("No product ID in URL.");
5
6   showArtisanSkeleton();
7   // Fetch artisan info & their products/collections
8   const artisanData = await fetchArtisanByProduct(productId);
9   const artisanID = artisanData.artisanId; // Display artisan info at the top
10   hideArtisanSkeleton();
11   displayArtisanInfo(artisanData);
12
13   const artisanProducts = await fetchArtisanProduct(artisanID);
14   const artisanCollections = await fetchArtisanCollection(artisanID);
15   displayProducts(artisanProducts);
16
17   // Set up tab switching
18   setupCategorySwitching(artisanProducts, artisanCollections);
19 });
20
21 // Commission button
22 document.addEventListener("DOMContentLoaded", () => {
23   const commissionBtn = document.querySelector(".commission");
24
25   if (commissionBtn) {
26     commissionBtn.addEventListener("click", () => {
27       window.location.href = `commission.html?artisanId=${window.artisanID}`;
28     });
29   }
30 });
31
32 function showArtisanSkeleton() {
33   const profilePic = document.querySelector(".profilePic img");
34   const name = document.querySelector(".name");
35   const username = document.querySelector(".username");
36
37   // Hide skeleton classes
38   profilePic.classList.add("skeleton-circle");
39   name.classList.add("skeleton-line");
40   username.classList.add("skeleton-line");
41
42   // Hide the real image temporarily
43 }

```



```

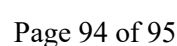
wishList.html X
1 <doctype html>
2 <html lang="en">
3 <head>
4 <meta charset="UTF-8" />
5 <meta name="viewport" content="width=device-width" />
6 <title>Wishlist</title>
7 <link rel="stylesheet" href="css/wishlist.css" />
8 </head>
9
10 <body>
11 <!-- Page header with back and home and cart buttons -->
12 <div class="page-header">
13 <a href="#wishlistHistory.back()" class="back icon" location.href="/index.html">
14 
15 </a>
16 <div class="side icons">
17 
18 
19 </div>
20 </div>
21
22 <!-- Main content area for wishlist page -->
23 <div class="wishlist-page">
24
25 <!-- Wishlist title -->
26 <div class="wishlist-title">
27 <h2>Wishlist</h2>
28 </div>
29
30 <!-- Tabs -->
31 <div class="wishlist-tabs">
32 <div class="wishlist-tab">
33 <a href="#products" data-tab="products" class="active">Products</a>
34 <a href="#collections" data-tab="collections">Collections</a>
35 </div>
36
37 <div id="indicator" class="indicator"></div>
38 </div>
39
40 <!-- Message and button -->
41 <div class="wishlist-long-line">
42 <div>
43
44 <!-- Message and button -->
45 <div class="login-promot">
46 <p>Please login to view your wishlist</p>
47 <button onclick="location.href='login.html'">login</button>
48 </div>
49
50 <!-- Product wishlist -->

```

```

commission.html X
1 <doctype html>
2 <html lang="en">
3 <head>
4 <meta charset="UTF-8" />
5 <title>Commission Page</title>
6 <meta name="viewport" content="width=device-width, initial-scale=1.0" />
7
8 <link rel="stylesheet" href="css/commission.css" />
9 <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/4.7.0/css/font-awesome.min.css" />
10 <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/5.6.0/css/all.min.css" />
11 </head>
12
13 <body>
14
15 <div>
16
17 <div class="section">
18
19 <div class="header">
20 <a href="#back" class="back icon" onclick="history.back()">
21 <div class="fa fa-angle-left"></div>
22 </a>
23 </div>
24
25 <div class="Profile-section">
26 <div class="Commission For"></div>
27 <div class="artist-name"></div>
28 <div class="Profile-picture">
29 
30 </div>
31
32 <div class="steps-progress">
33
34 <div class="person-info">
35 <div class="person-icon"></div>
36 <div class="person-text">
37 <div class="Person Details"></div>
38 </div>
39
40 <div class="line" id="line"></div>
41 </div>
42
43 <div class="order-info" class="hidden">
44

```



```

1 public > @ Login.html > @html > @body > @divwrap > @pam
2 <html lang="en">
3 <head>
4 <meta charset="UTF-8">
5 <meta name="viewport" content="width=device-width, initial-scale=1.0">
6 <title>Document</title>
7 <link rel="stylesheet" href="css/login.css" />
8 <link href="https://cdn.jsdelivr.net/gh/3.0.3/fonts/basic/boxicons.min.css" rel="stylesheet">
9 <link rel="preconnect" href="https://fonts.gstatic.com" />
10 <link href="https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700;800;900;1100" />
11 </head>
12 <body>
13 <div class="shape">
14 <div class="img" src="assets/wave_login" />
15 </div>
16 <div class="text">
17 <p class="m">Welcome Back!</p>
18 <p class="m">Continue discovering places made with </p>
19 <p class="m">purpose, and designed to inspire.</p>
20 </div>
21 <div class="login">
22 <div class="form">
23 <div class="input">
24 <input type="text" id="login-username" required>
25 <label>Username</label>
26 </div>
27 <div class="input">
28 <input type="password" id="login-password" required>
29 <label>Password</label>
30 </div>
31 <div class="input">
32 <input type="checkbox" checked="" />
33 <label>Remember me</label>
34 </div>
35 <div class="input">
36 <input type="button" value="Login" />
37 </div>
38 <div class="reg-link">
39 <a href="#">Sign Up</a>
40 </div>
41 </div>
42 </div>
43 </body>
44 </html>

```

```

1 public > @ Login.html >
2 const shape = document.querySelector('.shape');
3 const loginForm = document.querySelector('.login-form');
4 const loginLink = document.querySelector('.login-link');
5
6
7
8 const registerBtn = document.querySelector('.reg-link');
9 const loginBtn = document.querySelector('.login-link');
10
11 const welcome = document.querySelector('.welcome');
12 const loginForm = document.querySelector('.login-form');
13
14 const timeline = gsap.timeline({ paused: true });
15
16 const large = window.matchMedia("(max-width: 1700px)");
17 const mid = window.matchMedia("(max-width: 768px)");
18 const small = window.matchMedia("(max-width: 480px)");
19
20
21 if (small.matches) {
22 console.log("Screen is 480px or smaller.");
23
24 timeline.to(shape, { scale: 4, duration: .7, ease: 'sine.inOut' })
25 .to(loginForm, { opacity: 0, duration: 0.85 })
26 .to(loginLink, { opacity: 1, duration: 0.85 })
27 .to(shape, { x: '-110%', scale: 1, y: '-150%', duration: .7, ease: 'sine.inOut' });
28
29 } else if (mid.matches) {
30 console.log("Medium screen (576px)");
31
32 timeline.to(shape, { scale: 4.3, duration: .7, ease: 'sine.inOut' })
33 .to(loginForm, { opacity: 0, duration: 0.85 })
34 .to(loginLink, { opacity: 1, duration: 0.85 })
35 .to(shape, { x: '-130%', scale: 1, y: '-140%', duration: .7, ease: 'sine.inOut' });
36
37 } else if (large.matches) {
38 console.log("Viewport is 1700px or smaller.");
39
40 timeline.to(welcome, { opacity: 0, x: 30, duration: .7, stagger: 2 })
41 .to(shape, { scale: 1, duration: .7, ease: 'sine.inOut' })
42 .to(loginForm, { opacity: 0, duration: 0.85 })
43 .to(loginLink, { opacity: 1, duration: 0.85 })
44 .to(shape, { x: '-180%', scale: 1, duration: .7, ease: 'sine.inOut' })
45 .to(loginForm, { opacity: 1, x: 30, duration: .7, stagger: 2 });
46
47
48
49

```